FastAPI in Production

FastAPI in Production: Build an Order Platform, One Reliability Pattern at a Time

Format: core day (Labs 0-8, 7.5 hours) + optional advanced day (Labs 9-14, 6.5 hours)
Audience: AI engineers comfortable with Python and familiar with basic FastAPI routing
Runtime: Docker Compose locally; Kubernetes + Istio concepts and manifests for production

> [!TIP] **Use this as a workbook, not a document to read end-to-end.** For every lab: run the happy path, inject the stated failure, observe it through logs/metrics, apply the pattern, and prove the recovery.

Workshop control panel

Jump to a lab

Foundation	Reliability	Distributed workflow	Platform and operations
Lab 0: bootstrap	Lab 3: dependency failure	Lab 5: async messaging	Lab 8: Kubernetes and mesh
Lab 1: FastAPI boundary	Lab 4: gateway and discovery	Lab 6: saga	Lab 9: admission control
Lab 2: stateless writes	Lab 10: messaging operations	Lab 7: CQRS and event sourcing	Labs 11-14: operate at scale

Learning path

```
%% Learning path from a simple endpoint to a full reliability
platform.
flowchart TD
  A[Simple FastAPI endpoint] --> B[Durable, idempotent order write]
  B --> C[Bounded downstream calls]
  C --> D[Gateway and discovery]
  D --> E[Outbox and async events]
  E --> F[Saga and compensations]
  F --> G[Read models and event history]
  G --> H[Mesh, security, observability and safe delivery]
```

How to work each lab

```
%% Per-lab loop: run, break, observe, fix, verify.
flowchart LR
  R[1. Read the failure] --> U[2. Run the baseline]
  U --> B[3. Break it deliberately]
```

```
  B --> O[4. Observe evidence]
  O --> F[5. Apply the pattern]
  F --> V[6. Verify and reflect]
```

Step	Learner action	Evidence to capture
Run	Execute the provided command or endpoint.	Response, record count, message offset, or trace ID.
Break	Introduce the controlled failure exactly once.	Timeout, duplicate command, restart, invalid event, or failed deployment.
Observe	Inspect logs, metrics, queue age, or database state.	What actually happened—not what you expected.
Fix	Apply the smallest pattern that contains the failure.	Code/configuration diff.
Verify	Repeat the same failure.	Expected safe behavior and no duplicate side effects.

> [!IMPORTANT] Do not mark a lab complete because the code starts. Mark it complete only after you can explain the failure mode, demonstrate it, and show why the pattern changes the outcome.

Progress board

Copy this checklist into your own notes and check items as you complete the evidence, not merely the reading.

- ☐ Lab 0 - service starts, stops, and exposes correctly scoped health checks
- ☐ Lab 1 - invalid input and blocking I/O have been demonstrated
- ☐ Lab 2 - concurrent duplicate order requests persist one order
- ☐ Lab 3 - timeout, bulkhead, and circuit-breaker recovery have been observed
- ☐ Lab 4 - traffic reaches the service through gateway/discovery rather than a Pod address
- ☐ Lab 5 - outbox recovers an event after a simulated crash; consumer ignores a duplicate
- ☐ Lab 6 - a compensation runs after a downstream saga step fails
- ☐ Lab 7 - query projection lag and aggregate version conflict have been explained
- ☐ Lab 8 - a Pod rollout, readiness, and mesh policy have been inspected
- ☐ Labs 9-14 - traffic pressure, DLQ, trace, authorization, canary, and cache failure scenarios have been completed

▶ **Before starting: how do I know which pattern to use?**

What learners build

By the end, learners have evolved a single `order-service` into a small but realistic order platform.

```
%% Target system: order service and its downstream dependencies.
flowchart LR
  C[Web / AI client] --> G[API Gateway]
  G --> O[Order service]
  O --> P[PostgreSQL]
  O <--> I[Inventory service]
  O <--> PY[Payment service]
  O --> K[(Kafka / Redpanda)]
  K --> S[Shipping service]
```

```
K --> N[Notification service]
O -. telemetry .-> T[OpenTelemetry collector]
```

The business flow is intentionally familiar: **place an order**. A customer buys items; inventory must be reserved, payment authorized, shipment created, and a confirmation sent. Those operations live in different services and can fail independently. Each lab adds one production capability because the earlier, simpler version has a concrete failure.

> Scope note: FastAPI is an HTTP application framework. API gateway, service discovery, and a service mesh are platform responsibilities. A production backend needs to cooperate with them cleanly instead of reimplementing them in Python.

---

1. Core-day plan (Labs 0-8)

Time	Lab	The pain we expose	What learners add
09:30–10:00	0. Bootstrap	"It works on my laptop"	repeatable local stack, health checks, environment-based settings
10:00–10:50	1. Production-ready FastAPI core	invalid requests, blocking handlers, opaque errors	app factory, lifespan, validation, async I/O, error contract
11:00–11:45	2. Stateless order API	duplicate requests and lost state on restart	PostgreSQL, idempotency, transactions, migrations
11:45–12:30	3. Synchronous dependencies	payment or inventory makes checkout hang	timeouts, retries, circuit breaker, bulkhead
13:15–14:05	4. Gateway and discovery	clients know too much about services	API gateway, service discovery, zero-trust boundaries
14:05–15:05	5. Async messaging	response path is too long; events get lost	transactional outbox, Kafka/Redpanda, idempotent consumer
15:15–16:00	6. Distributed consistency	payment succeeds but stock/shipping fails	saga orchestration and compensations
16:00–16:40	7. CQRS + event sourcing	order history/query needs diverge	projections, event store, replay
16:40–17:20	8. Kubernetes, sidecar, mesh	per-service networking/telemetry policy drift	Deployment, HPA, Istio sidecar, traffic policy
17:20–17:30	Capstone	prove reliability	failure game day and review

1.1 Environment contract — exactly where each lab runs

This course deliberately uses **two environments**, in sequence. Do not attempt to introduce Kubernetes or a service mesh while learning the basic request/data patterns. It makes failures harder to attribute.

Course phase	Labs	Runtime learners use	Discovery and routing mechanism	What is intentionally out of scope
Local foundation	0-7, 9-11, 13-14	Docker Compose plus local Python tools	Compose DNS: `http://order-service:8000`; Kong at `http://localhost:8000`	Kubernetes, Istio, cloud load balancers
Platform translation	8 and the Kubernetes parts of 12-13	`kind` or `minikube` plus Istio	Kubernetes `Service` + CoreDNS; ingress/gateway; Envoy sidecar	Recreating every local lab inside a cluster

```
%% Environment split: Compose on a laptop (Labs 0–7) vs. Kubernetes +
Istio (Lab 8+).
flowchart LR
  subgraph Local["Labs 0–7: Docker Compose on your laptop"]
    LClient[Client] --> LKong[Kong :8000]
    LKong --> LOrder[order-service]
    LOrder --> LInventory[inventory-service]
    LOrder --> LPayment[payment-service]
    LOrder --> LDB[(Postgres)]
  end
  Local -. "Lab 8: translate the same boundary" .-> Cluster
  subgraph Cluster["Labs 8+: Kubernetes + Istio"]
    KClient[Client] --> KIngress[Ingress / Gateway]
    KIngress --> KSvc[Kubernetes Service]
    KSvc --> KPod[Order Pod + Envoy sidecar]
    KPod --> KMesh[Internal services via mesh]
  end
```

> [!NOTE] **Lab 4 is a Docker Compose lab.** It runs Kong locally and uses Compose DNS for service discovery. Lab 8 later maps that same idea to Kubernetes `Service`, CoreDNS, readiness endpoints, ingress, and Istio. A Kubernetes manifest shown before Lab 8 is explanatory only; do not apply it yet.

1.2 Code-reading contract

The guide uses three snippet kinds. This prevents a learner from assuming every short example is a complete production repository.

Label	Meaning	What you should do
RUN	Complete enough to execute after the stated setup.	Copy it into the named file and run the verification command.
WIRE	Shows how two pieces connect; surrounding repository code is referenced.	Add it after the earlier lab files exist.
CONCEPT	Explains a production design choice or a platform policy.	Read it, then use the exercise to validate the behavior.

Unless explicitly marked **CONCEPT**, code is written to be copied as a starting point. Production systems still need tests, secret injection, pinned image versions, security review, and organization-specific platform configuration.

1.3 Self-learning success loop

At every lab, keep this evidence in a `notes.md` file: command run, expected output, observed failure, applied change, and retest output. This turns the course into an auditable engineering exercise rather than passive reading.

```
# Lab evidence files kept under notes/, one per lab.
notes/
  lab-02-idempotency.md     # duplicate request evidence and database
query result
  lab-03-resilience.md      # timeout/breaker metrics and recovery
observation
  lab-05-outbox.md          # crash simulation and eventual event
delivery
  lab-08-platform.md        # rollout/readiness/mesh evidence
```

2. Before the workshop

Install

- Python 3.12+ and `uv` (or pip + virtualenv)
- Docker Desktop with Compose
- `kubectl`; optionally `kind` or `minikube`
- A local Kubernetes cluster and Istio only for Lab 8

Create the workspace

```
# Scaffold the project folder and install dependencies with uv.
mkdir -p fastapi-production-workshop/{services,infra,k8s}
cd fastapi-production-workshop
uv init
uv add fastapi 'uvicorn[standard]' sqlalchemy asyncpg alembic \
  httpx tenacity aiobreaker structlog prometheus-fastapi-
        instrumentator \
  opentelemetry-api opentelemetry-sdk opentelemetry-exporter-otlp \
  opentelemetry-instrumentation-fastapi opentelemetry-instrumentation-
        httpx \
  aiokafka
uv add --dev pytest pytest-asyncio respx ruff
```

All application code lives under `services/order_service/app/`. Every file path in this workshop is relative to the repository root and uses this prefix. Do not create a second `app/` package at the repository root. The app factory is `services/order_service/app/main.py`, and the server runs as `services.order_service.app.main:app`.

Create the directories first, then the package marker files, so imports work the same from a terminal, a test runner, and a container. `touch` does not create missing parent folders, so `mkdir -p` must run before it:

```
# Create the service package folders and __init__.py markers.
mkdir -p
        services/order_service/app/{api,application,domain,infrastructure}
        \
  services/mock_dependency infra/gateway tests/contracts notes
touch services/__init__.py services/order_service/__init__.py \
  services/order_service/app/__init__.py \
```

```
  services/order_service/app/{api,application,domain,infrastructure}/__init__.py \
  services/mock_dependency/__init__.py
```

Create `.env.example`. Labs 0-3 run FastAPI on the host, so Kafka uses `localhost`; the host process cannot resolve the Compose name `redpanda`. Lab 4 uses `infra/.env.compose` for container hostnames.

```
# Host-process settings for Labs 0–3 (localhost hostnames).
DATABASE_URL=postgresql+asyncpg://orders:orders@localhost:5432/orders
INVENTORY_URL=http://localhost:8002
PAYMENT_URL=http://localhost:8003
KAFKA_BOOTSTRAP_SERVERS=localhost:9092
```

Ignore it, then copy it:

```
# Git-ignore .env and create your local copy.
echo ".env" >> .gitignore
cp .env.example .env
```

> [!WARNING] `.env` is for local learning only and is ignored by Git. Production credentials belong in a secret manager and are injected at runtime; never commit passwords or tokens.

Suggested layout after Lab 5:

```
# Target repository layout; built up across labs, not all at once.
.env.example
.env                            # copied from .env.example; git-ignored
pyproject.toml
services/
  order_service/
    Dockerfile
    alembic/
    app/
      main.py                   # create_app() and module-level app
      config.py
      observability.py          # Lab 8
      api/                      # routes and error handlers
      application/              # use cases
      domain/
      infrastructure/           # Postgres, HTTP, Kafka adapters
  mock_dependency/
  inventory_service/
  payment_service/
  shipping_service/
infra/
  compose.yaml
  .env.compose
  sql/
  gateway/                      # Kong/Envoy config
k8s/
  base/                         # manifests
  istio/                        # DestinationRule, VirtualService
tests/
  contracts/
notes/
```

Why this layout?

Routes only translate HTTP into an application command. Domain rules do not import FastAPI, SQLAlchemy, or Kafka. Infrastructure adapts a port to Postgres/HTTP/Kafka. This keeps business logic testable and lets infrastructure change without a rewrite.

Lab 0 — Bootstrap a service that can start and stop safely

Goal: establish the non-negotiable baseline used by all later labs.

Step 0.1: configuration belongs in environment, not source code

Problem: hard-coded database URLs and hostnames work on one laptop and fail everywhere else, and credentials end up in Git.

Pattern: read configuration from environment variables once, cache it, and fail at startup when a required value is missing.

Create `services/order_service/app/config.py`:

```
# Environment-based settings, loaded once and cached per process.
import os
from functools import lru_cache


class Settings:
    def __init__(self):
        self.app_name = os.getenv("APP_NAME", "order-service")
        self.environment = os.getenv("ENVIRONMENT", "local")
        self.database_url = os.environ["DATABASE_URL"]
        self.inventory_url = os.getenv("INVENTORY_URL",
        "http://inventory-service:8000")
        self.payment_url = os.getenv("PAYMENT_URL", "http://payment-
        service:8000")
        self.kafka_bootstrap_servers =
        os.getenv("KAFKA_BOOTSTRAP_SERVERS", "redpanda:9092")

@lru_cache
def get_settings():
    return Settings()
```

Verify: check that a missing value fails and a provided value loads.

```
# Without DATABASE_URL: expect KeyError: 'DATABASE_URL'
env -u DATABASE_URL uv run python -c "from
        services.order_service.app.config import get_settings;
        get_settings()"

# With DATABASE_URL: expect the URL printed back
DATABASE_URL=postgresql+asyncpg://orders:orders@localhost:5432/orders
        \
  uv run python -c "from services.order_service.app.config import
        get_settings; print(get_settings().database_url)"
```

In Lab 4, the app moves into Compose and the hostname changes to `postgres`; the two environment files make that explicit.

Step 0.2: use an app factory and lifespan

Problem: if the pool and clients are created at import time or per request, the service either starts broken, leaks connections, or cannot shut down cleanly.

Pattern: an app factory builds the application. A lifespan hook creates shared resources once at startup and closes them at shutdown. `/livez` answers "is the process alive?"; `/readyz` answers "can it take traffic?"

```
# services/order_service/app/main.py
# App factory with lifespan hooks and liveness/readiness endpoints.
from contextlib import asynccontextmanager
from fastapi import FastAPI
from .config import get_settings


@asynccontextmanager
async def lifespan(app):
    settings = get_settings()
    # Create expensive shared clients/pools here. Do not create one
        per request.
    app.state.settings = settings
    yield
    # Close pools/producers here after Kubernetes removes this Pod
        from endpoints.


def create_app():
    app = FastAPI(title="Order Service", version="0.1.0",
        lifespan=lifespan)

    @app.get("/livez", include_in_schema=False)
    async def livez():
        return {"status": "alive"}

    @app.get("/readyz", include_in_schema=False)
    async def readyz():
        # Lab 2 checks DB; Lab 5 checks whether messaging can publish.
        return {"status": "ready"}
    return app


app = create_app()
```

Run it:

```
# Start the service with uvicorn and check liveness.
uv run uvicorn services.order_service.app.main:app --env-file .env --
        reload --port 8000
curl localhost:8000/livez
```

Verify:

```
curl -s localhost:8000/livez    # expect {"status":"alive"}
curl -s localhost:8000/readyz   # expect {"status":"ready"}
# Stop uvicorn with Ctrl+C: it should exit after the lifespan
        shutdown, without a traceback.
```

Failure clinic

Symptom	Why it happens	Fix
`RuntimeWarning: coroutine was never awaited`	an async startup/cleanup function was called like normal Python	call it with `await` inside lifespan
a new database/HTTP client appears in every request	connection setup was placed inside a route/dependency without caching	create one pool/client in lifespan and close it on shutdown

Symptom	Why it happens	Fix
`asyncio.run() cannot be called from a running event loop`	the ASGI server already owns the loop	make the function `async` and `await` it
readiness succeeds while DB is unavailable	`/readyz` only checks process liveness	make readiness a lightweight dependency check; keep `/livez` process-only

Checkpoint: explain why a liveness probe must not query Postgres. If Postgres is down, restarting every otherwise healthy Pod amplifies the outage.

Lab 1 — Build a predictable FastAPI boundary

Problem: a demo endpoint accepts anything, returns inconsistent exceptions, and blocks the event loop under load.

Step 1.1: make the HTTP contract explicit

Problem: the endpoint accepts any JSON. Bad values reach business code and the database, and hand-written checks drift away from what the API documents.

Pattern: Pydantic models declare the contract once. FastAPI validates at the HTTP boundary and generates OpenAPI from the same types.

Request bodies are declared as Pydantic models. FastAPI validates every field before the handler runs, generates OpenAPI documentation from the same models, and passes the handler typed objects instead of raw dictionaries.

```python
# services/order_service/app/api/orders.py
# Step 1.1: Pydantic request models and a stub POST /v1/orders
        endpoint.
from uuid import UUID
from decimal import Decimal
from fastapi import APIRouter, Header, status
from pydantic import BaseModel, Field

router = APIRouter(prefix="/v1/orders", tags=["orders"])

class OrderLine(BaseModel):
    sku: str = Field(min_length=1)
    quantity: int = Field(gt=0)
    unit_price: Decimal = Field(gt=0)

class CreateOrderRequest(BaseModel):
    customer_id: UUID
    lines: list[OrderLine] = Field(min_length=1)

@router.post("", status_code=status.HTTP_202_ACCEPTED)
async def create_order(
    body: CreateOrderRequest,
    idempotency_key: str = Header(min_length=16, max_length=128),
):
    # The application service is introduced in Lab 2.
    return {"order_id": "created-in-lab-2", "status": "PENDING"}
```

Key decisions:

- Request and response data are separate. Never serialize ORM objects directly.
- Money is `Decimal`, never `float`.
- An order may complete asynchronously, so return `202 Accepted` with a stable order ID.
- Idempotency is a protocol feature, not a UI feature. Mobile retries, gateways, and clients can all repeat a POST.

Step 1.2: return one error shape

Problem: each failure returns a different shape. Clients cannot write one error handler, and internal exception text leaks to callers.

Pattern: one exception type for business errors and one handler for validation errors, both returning the same JSON structure. The trace ID is attached at the boundary, so Lab 8 can replace it with the real OpenTelemetry trace ID.

```
# services/order_service/app/api/errors.py
# Step 1.2: domain and validation exception handlers returning one
        error shape.
from uuid import uuid4

from fastapi import FastAPI
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse


class DomainError(Exception):
    def __init__(self, code, message, status_code=409):
        self.code, self.message, self.status_code = code, message,
        status_code


def install_error_handlers(app: FastAPI):
    @app.exception_handler(DomainError)
    async def domain_error(_, exc):
        trace_id = str(uuid4())  # In Lab 8 obtain current OTEL trace
        ID instead.
        return JSONResponse(
            status_code=exc.status_code,
            content={"code": exc.code, "message": exc.message,
        "trace_id": trace_id},
        )

    @app.exception_handler(RequestValidationError)
    async def validation_error(_, exc):
        trace_id = str(uuid4())  # In Lab 8 obtain current OTEL trace
        ID instead.
        return JSONResponse(
            status_code=422,
            content={
                "code": "VALIDATION_ERROR",
                "message": "Request validation failed",
                "trace_id": trace_id,
                "details": [
                    {"field": ".".join(map(str, e["loc"])), "message":
        e["msg"]}
                    for e in exc.errors()
                ],
            },
        )
```

Register the router and handlers in `create_app()`. Then send invalid data:

```
# services/order_service/app/main.py — add these imports
# Register the error handlers and router in create_app().
from .api.errors import install_error_handlers
from .api.orders import router as orders_router

# services/order_service/app/main.py — add these two lines inside
        create_app(), before return app
install_error_handlers(app)
app.include_router(orders_router)

# Send an invalid request to confirm the 422 error shape.
curl -i -X POST localhost:8000/v1/orders \
  -H 'content-type: application/json' -H 'Idempotency-Key: a-very-
        long-client-generated-key' \
  -d '{"customer_id":"not-a-uuid","lines":[]}'
```

FastAPI's 422 validation response is useful, but standardize it too if your API has an external contract.

Verify Steps 1.1 and 1.2: the server from Lab 0 must be running. An invalid body returns the shared error shape, and a valid body returns `202`.

```
curl -s -X POST localhost:8000/v1/orders -H 'content-type:
        application/json' \
  -H 'Idempotency-Key: a-very-long-client-generated-key' \
  -d '{"customer_id":"not-a-uuid","lines":[]}' | python3 -m json.tool
# expect "code": "VALIDATION_ERROR", a "trace_id", and "details"
        naming customer_id and lines

curl -s -o /dev/null -w "%{http_code}\n" -X POST
        localhost:8000/v1/orders \
  -H 'content-type: application/json' -H 'Idempotency-Key: a-very-
        long-client-generated-key' \
  -d '{"customer_id":"11111111-1111-1111-1111-111111111111","lines":
        [{"sku":"SKU-001","quantity":1,"unit_price":"49.90"}]}'
# expect 202
```

Step 1.3: understand async is not magic

Problem: one blocking call inside `async def` (for example `time.sleep`) freezes the event loop, so unrelated requests on that worker wait too.

Pattern: use async clients for I/O, so the loop can serve other requests while one waits. Send CPU-heavy work to worker processes, not the event loop.

```
# Blocking vs. non-blocking handlers under async def.
# Wrong: blocks every request sharing this event loop worker.
@router.get("/wrong")
async def wrong():
    import time
    time.sleep(2)
    return {"ok": True}

# Right for network I/O: the HTTP client yields while waiting.
import httpx
@router.get("/right")
async def right():
    async with httpx.AsyncClient(timeout=1.0) as client:
        response = await client.get("https://example.internal/health")
    return response.json()
```

For CPU-heavy tokenization, document parsing, or local model inference, use a worker system (Celery/Arq/Temporal/Kubernetes Job) or a process pool. `async def` does not make CPU work parallel.

Verify: add the two handlers temporarily to `services/order_service/app/api/orders.py`, so they are served at `/v1/orders/wrong` and `/v1/orders/right`. In `/right`, point the URL at `http://localhost:8000/livez`, since `example.internal` does not resolve.

```
# Blocking: /livez waits behind the 2-second time.sleep in /wrong
curl -s localhost:8000/v1/orders/wrong > /dev/null & sleep 0.2
time curl -s localhost:8000/livez
# expect roughly 2 seconds

# Non-blocking: /livez answers immediately while /right waits on httpx
curl -s localhost:8000/v1/orders/right > /dev/null & sleep 0.2
time curl -s localhost:8000/livez
# expect well under 0.5 seconds
```

Remove both temporary handlers after the check.

Failure clinic

Symptom	Root cause	Fix
p99 latency jumps whenever one request calls `time.sleep`, `requests`, or a blocking SDK	the event loop is blocked	use an async library; otherwise run blocking work in a bounded threadpool or external worker
`422` has fields clients cannot rely on	default validator detail leaked as the API contract	add a validation exception handler and publish your `Problem` shape
duplicate / missing headers in an AI agent client	headers were not enforced by contract	explicitly model `Header(...)`; reject absent idempotency keys for unsafe operations
`float` price is `19.989999...`	binary floating-point	accept/compute money as `Decimal`, serialize a documented string/decimal form

Lab 2 — Keep services stateless, make writes durable and idempotent

Problem: an in-memory `dict` loses orders after a restart and concurrent retries create two orders.

Step 2.1: define persistence boundaries

Problem: the in-memory `dict` loses every order on restart, and each replica keeps its own copy, so the same request can succeed on one replica and miss on another.

Pattern: store orders in Postgres. Each process owns one connection pool with a fixed size, and each request gets its own session from that pool.

```python
# services/order_service/app/infrastructure/db.py
# Creates one connection pool per process and a per-request session
        dependency.
# pool_pre_ping drops dead connections; expire_on_commit=False avoids
        lazy loads in async code.
from sqlalchemy.ext.asyncio import async_sessionmaker,
        create_async_engine
from ..config import get_settings

engine = create_async_engine(
    get_settings().database_url,
    pool_pre_ping=True, pool_size=10, max_overflow=20,
)
SessionFactory = async_sessionmaker(engine, expire_on_commit=False)

async def get_session():
    async with SessionFactory() as session:
        yield session
```

For the first local run, save this as `infra/sql/001_orders.sql`. The course executes it directly so you can focus on the behavior. In a production repository, commit the same change as an Alembic migration; Lab 13 covers the safe evolution rule.

```sql
-- Core tables for orders and idempotency keys (Lab 2).
CREATE TABLE orders (
  id UUID PRIMARY KEY,
  customer_id UUID NOT NULL,
  status TEXT NOT NULL,
  total NUMERIC(10,2) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE idempotency_keys (
  key TEXT PRIMARY KEY,
  request_hash TEXT NOT NULL,
  order_id UUID NOT NULL REFERENCES orders(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Step 2.2: enforce idempotency atomically

Problem: a client retries a `POST` after a timeout. If the code first checks whether the key exists and then inserts, two concurrent retries can both pass the check and create two orders.

Pattern: store the idempotency key under a unique constraint and let the insert decide the winner inside one transaction. The losing request reads the existing result instead of creating an order.

```python
# services/order_service/app/application/create_order.py
# Step 2.2: atomic idempotent order insert in one transaction.
from hashlib import sha256
from json import dumps
from uuid import uuid4
from decimal import Decimal
from sqlalchemy import text
from ..api.errors import DomainError

async def create_order(session, command, idempotency_key):
    # `command` is `CreateOrderRequest.model_dump(mode="json")` from
        the Lab 1 route.
    payload = command
    request_hash = sha256(dumps(payload,
        sort_keys=True).encode()).hexdigest()
```

```
    order_id = uuid4()

    async with session.begin():
        result = await session.execute(text("""
          INSERT INTO idempotency_keys(key, request_hash, order_id)
          VALUES (:key, :hash, :order_id)
          ON CONFLICT (key) DO NOTHING
          RETURNING order_id
        """), {"key": idempotency_key, "hash": request_hash,
        "order_id": order_id})
        inserted = result.scalar_one_or_none()

        # ON CONFLICT DO NOTHING returned no row: the key already
        exists.
        # Same request hash = safe retry, return the original order.
        Different hash = key reused for another payload.
        if inserted is None:
            existing = await session.execute(text(
              "SELECT request_hash, order_id FROM idempotency_keys
        WHERE key = :key"),
              {"key": idempotency_key})
            old_hash, old_order_id = existing.one()
            if old_hash != request_hash:
                raise DomainError("IDEMPOTENCY_KEY_REUSED", "Key
        belongs to another request", 422)
            return old_order_id, True

        total = sum(
            Decimal(str(line["unit_price"])) * int(line["quantity"])
            for line in command["lines"]
        )
        await session.execute(text("""
          INSERT INTO orders(id, customer_id, status, total)
          VALUES (:id, :customer_id, 'PENDING', :total)
        """), {"id": order_id, "customer_id": command["customer_id"],
        "total": total})
    return order_id, False
```

The unique key is the concurrency control. Checking first with `SELECT` is race-prone: two requests can both observe "missing."

Step 2.2b: replace the Lab 1 stub with the real write path

Problem: the Lab 1 endpoint returns a fake order and never touches the database or the idempotency logic.

Pattern: the route takes a database session from a dependency and passes the validated body to the application function, so HTTP, transaction, and business rules each stay in one place.

In `services/order_service/app/api/orders.py`, replace the temporary Lab 1 endpoint with this version. This is the missing wiring between HTTP, the database session, and the application function above.

```
# Step 2.2b: real write path wired to the DB session and
        create_order().
from fastapi import APIRouter, Depends, Header, status
from ..infrastructure.db import get_session
from ..application.create_order import create_order as persist_order

# CreateOrderRequest and OrderLine are unchanged from Step 1.1.
router = APIRouter(prefix="/v1/orders", tags=["orders"])
```

```
@router.post("", status_code=status.HTTP_202_ACCEPTED)
async def create_order_endpoint(
    body: CreateOrderRequest,
    idempotency_key: str = Header(min_length=16, max_length=128),
    session = Depends(get_session),
):
    order_id, replayed = await persist_order(
        session, body.model_dump(mode="json"), idempotency_key
    )
    return {
        "order_id": str(order_id),
        "status": "PENDING",
        "idempotent_replay": replayed,
    }
```

Now run the same request twice. The first response has `idempotent_replay: false`; the second has `true` and the same `order_id`.

Verify the concurrency guarantee: send five identical requests in parallel, then send the same key with a different body.

```
ORDERS_BEFORE=$(docker compose -f infra/compose.yaml exec -T postgres
        psql -U orders -d orders -tAc "select count(*) from orders;")
for i in 1 2 3 4 5; do
  curl -s -o /dev/null -w "%{http_code} " -X POST
        localhost:8000/v1/orders -H 'content-type: application/json' \
    -H 'Idempotency-Key: race-demo-key-0001' \
    -d '{"customer_id":"11111111-1111-1111-1111-111111111111","lines":
        [{"sku":"SKU-001","quantity":1,"unit_price":"49.90"}]}' &
done; wait; echo
ORDERS_AFTER=$(docker compose -f infra/compose.yaml exec -T postgres
        psql -U orders -d orders -tAc "select count(*) from orders;")
echo "new orders: $((ORDERS_AFTER - ORDERS_BEFORE))"   # expect 1

# Same key, different cart: expect 422 IDEMPOTENCY_KEY_REUSED
curl -s -X POST localhost:8000/v1/orders -H 'content-type:
        application/json' \
  -H 'Idempotency-Key: race-demo-key-0001' \
  -d '{"customer_id":"11111111-1111-1111-1111-111111111111","lines":
        [{"sku":"SKU-002","quantity":3,"unit_price":"10.00"}]}'
```

Step 2.3: run local dependencies

Problem: the schema must exist before the first request, and every learner's database must match.

Pattern: start Postgres and Redpanda in Compose and load the reviewed schema file explicitly. The application never creates tables on startup.

```
# infra/compose.yaml
# Local Postgres and Redpanda containers for Labs 0-3.
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: orders
      POSTGRES_PASSWORD: orders
      POSTGRES_DB: orders
    ports: ["5432:5432"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U orders -d orders"]
      interval: 5s
```

```
      timeout: 3s
      retries: 10
  redpanda:
    image: redpandadata/redpanda:latest
    command: redpanda start --overprovisioned --smp 1 --memory 1G --
        node-id 0 --check=false
    ports: ["9092:9092", "9644:9644"]

# Start dependencies, load the schema, and confirm the tables exist.
# Labs 0-3 run FastAPI on the host; only dependencies run in Compose.
docker compose -f infra/compose.yaml up -d postgres redpanda
docker compose -f infra/compose.yaml exec -T postgres \
  psql -U orders -d orders < infra/sql/001_orders.sql

# Confirm that the tables exist before starting the API.
docker compose -f infra/compose.yaml exec -T postgres \
  psql -U orders -d orders -c '\dt'
```

Expected result: `orders` and `idempotency_keys` appear in the table list.

Verify the connection pool: the app and the database can talk through the same engine.

```
uv run --env-file .env python -c "
import asyncio
from sqlalchemy import text
from services.order_service.app.infrastructure.db import engine

async def check():
    async with engine.connect() as conn:
        print((await conn.execute(text('select 1'))).scalar())

asyncio.run(check())
"
# expect 1
```

If the database is reset, rerun the same schema file. Do not make the application run DDL on startup; in a production service the deployment pipeline executes a reviewed migration before enabling new code.

Failure clinic

Symptom	Why	Fix
two orders for the same request under load	"read then insert" has a race	unique constraint + `INSERT ... ON CONFLICT` in one transaction
the same key with a changed cart silently returns the old order	key is treated only as a cache	store and compare a canonical request hash; return 422 on mismatch
`MissingGreenlet` from SQLAlchemy	ORM lazy-load triggered I/O outside `await`	load relations explicitly, use async APIs, or return DTOs before leaving the session
`too many connections` during a traffic spike	each worker has an unbounded/default pool	calculate pools across all replicas; cap `pool_size/max_overflow`; use PgBouncer when appropriate
Pods contain orders in memory	state was kept in globals/session	persist durable state; use Redis only for disposable cache/rate limits, not source of truth

Production rule: a stateless service may cache locally, but the correct response must not depend on which replica receives the request.

Lab 3 — Call other services without spreading an outage

Problem: checkout calls inventory and payment synchronously. When payment slows, each request holds resources waiting; retries multiply load and all order workers become unavailable.

Step 3.0: start deterministic local dependencies

Problem: a real payment provider cannot be made slow or broken on demand, so timeout and failure behavior cannot be repeated or tested.

Pattern: small mock services whose delay and failure mode come from environment variables. The same module runs locally and in Compose, so only the hostname changes.

Labs 0-3 run FastAPI on the host, so use two intentionally small mock services on your laptop. They make timeout and failure behavior repeatable without a real payment provider.

Create `services/mock_dependency/main.py`:

```
# Deterministic mock dependency with configurable delay and failure
        modes.
import asyncio
import os
from uuid import uuid4
from fastapi import FastAPI, HTTPException

app = FastAPI()

@app.get("/health")
async def health():
    return {"dependency": os.getenv("DEPENDENCY_NAME", "mock"),
        "status": "ok"}

@app.post("/authorizations")
async def authorize():
    delay = float(os.getenv("DELAY_SECONDS", "0"))
    if delay:
        await asyncio.sleep(delay)
    if os.getenv("FAIL_MODE") == "500":
        raise HTTPException(500, "Simulated payment outage")
    if os.getenv("FAIL_MODE") == "decline":
        raise HTTPException(402, "Simulated card decline")
    return {"authorization_id": str(uuid4())}

@app.post("/reservations")
async def reserve():
    return {"reservation_id": str(uuid4())}
```

Start the mocks in separate terminals:

```
# Run the two mock dependencies on separate ports and verify health.
# Terminal 1: inventory mock
DEPENDENCY_NAME=inventory uv run uvicorn
        services.mock_dependency.main:app --port 8002
```

```
# Terminal 2: payment mock
DEPENDENCY_NAME=payment uv run uvicorn
        services.mock_dependency.main:app --port 8003

# Terminal 3: verify both before running the order service
curl http://localhost:8002/health
curl http://localhost:8003/health
```

Expected result: each health endpoint returns its dependency name and `status: ok`. Lab 4 runs the exact same mock module as Compose services, so the only thing that changes is the hostname.

Verify the failure switch: restart the payment mock (terminal 2) with `FAIL_MODE=500`, then check it, then restart it without the variable.

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST
        localhost:8003/authorizations   # expect 500
curl -s -o /dev/null -w "%{http_code}\n" -X POST
        localhost:8003/authorizations   # after restart without
        FAIL_MODE: expect 200
```

Step 3.1: give every remote call a budget

Problem: a slow payment call holds the request open. Each retry adds load to a service that is already struggling, and the outage spreads to checkout.

Pattern: give every remote call a timeout. A circuit breaker counts dependency-availability failures; after too many, it opens and fails calls immediately without touching the network. After a cooldown it lets one probe through (half-open) and closes again only if that probe succeeds.

```python
# services/order_service/app/infrastructure/payment_client.py
# Step 3.1: payment client with timeouts, circuit breaker, and error
        mapping.
import httpx
from datetime import timedelta
from aiobreaker import CircuitBreaker

class PaymentDeclined(Exception):
    pass

class PaymentUnavailable(Exception):
    pass

payment_breaker = CircuitBreaker(
    fail_max=5,
    timeout_duration=timedelta(seconds=30),
    exclude=[PaymentDeclined],
)

class PaymentClient:
    def __init__(self, base_url, client):
        self.base_url, self.client = base_url, client

    @payment_breaker
    async def authorize(self, order_id, amount):
        try:
            response = await self.client.post(
                f"{self.base_url}/authorizations",
                json={"order_id": order_id, "amount": amount},
                headers={"Idempotency-Key": f"payment:{order_id}"},
```

```
                timeout=httpx.Timeout(connect=0.2, read=0.8,
        write=0.2, pool=0.2),
            )
            if 400 <= response.status_code < 500:
                raise PaymentDeclined("Payment was declined or request
        was invalid")
            if response.status_code >= 500:
                raise PaymentUnavailable(f"Payment returned
        {response.status_code}")
            return response.json()["authorization_id"]
        except httpx.TimeoutException as exc:
            raise PaymentUnavailable("Payment timed out") from exc
        except httpx.TransportError as exc:
            raise PaymentUnavailable("Payment transport failed") from
        exc
```

`CircuitBreaker` has three states:

- **closed**: normal calls; failures are counted.
- **open**: fail fast for a short period; protect both services.
- **half-open**: allow a small probe; close only if recovery succeeds.

Use retry only for *transient*, *safe/idempotent* calls. Give a request one end-to-end deadline; do not independently spend 1 second in each of five downstream calls.

Verify the timeout: with a 3-second delay, the call must fail at the 0.8-second read timeout, not after 3 seconds.

```
# Terminal 2: restart the payment mock with a 3-second delay and no
        FAIL_MODE
DELAY_SECONDS=3 DEPENDENCY_NAME=payment uv run uvicorn
        services.mock_dependency.main:app --port 8003

# Terminal 3: one call, timed
time uv run --env-file .env python - <<'EOF'
import asyncio, httpx
from services.order_service.app.infrastructure.payment_client import
        PaymentClient

async def main():
    async with httpx.AsyncClient() as http:
        try:
            await PaymentClient("http://localhost:8003",
        http).authorize("o-timeout", 10)
            print("ok")
        except Exception as exc:
            print(type(exc).__name__)

asyncio.run(main())
EOF
# expect PaymentUnavailable, and real time around 1 s (not 3 s)
```

Verify the circuit breaker: run the automated test first, then the manual check against the mock.

Create `tests/test_payment_breaker.py`:

```
# tests/test_payment_breaker.py
import httpx
import pytest
from aiobreaker import CircuitBreakerError

from services.order_service.app.infrastructure.payment_client import (
    PaymentClient,
    PaymentDeclined,
```

```
    PaymentUnavailable,
    payment_breaker,
)


class FixedStatusTransport:
    def __init__(self, status_code):
        self.status_code = status_code
        self.calls = 0

    async def post(self, url, **kwargs):
        self.calls += 1
        return httpx.Response(self.status_code,
         request=httpx.Request("POST", url))


@pytest.fixture(autouse=True)
def reset_breaker():
    payment_breaker.close()
    yield
    payment_breaker.close()


@pytest.mark.asyncio
async def test_breaker_opens_after_five_unavailable_responses():
    transport = FixedStatusTransport(500)
    client = PaymentClient("http://payment.com", transport)

    for _ in range(4):
        with pytest.raises(PaymentUnavailable):
            await client.authorize("order-1", 10)

    with pytest.raises(CircuitBreakerError):  # 5th failure opens the
        circuit
        await client.authorize("order-1", 10)

    calls_before = transport.calls
    with pytest.raises(CircuitBreakerError):  # open circuit: no
        request sent
        await client.authorize("order-1", 10)
    assert transport.calls == calls_before


@pytest.mark.asyncio
async def test_declines_never_open_the_breaker():
    transport = FixedStatusTransport(402)
    client = PaymentClient("http://payment", transport)

    for _ in range(10):
        with pytest.raises(PaymentDeclined):
            await client.authorize("o-1", 10)

uv run pytest tests/test_payment_breaker.py -v
# expect both tests PASSED
```

Manual check against the real mock: one Python process makes calls, so the breaker state lives for that run.

```
# Terminal 2: payment mock failing every authorization
FAIL_MODE=500 DEPENDENCY_NAME=payment uv run uvicorn
        services.mock_dependency.main:app --port 8003

# Terminal 3: calls 0-4 reach the mock and return PaymentUnavailable;
        calls 5-6 raise CircuitBreakerError
```

```
uv run --env-file .env python - <<'EOF'
import asyncio, httpx
from services.order_service.app.infrastructure.payment_client import
        PaymentClient

async def main():
    async with httpx.AsyncClient() as http:
        client = PaymentClient("http://localhost:8003", http)
        for i in range(7):
            try:
                await client.authorize(f"o-{i}", 10)
                print(i, "ok")
            except Exception as exc:
                print(i, type(exc).__name__)

asyncio.run(main())
EOF
# Terminal 2 log: only 5 "POST /authorizations 500" lines. Calls 5-6
        never reached the mock.
```

Recovery (half-open): this script waits 40 seconds after the circuit opens. Restart the mock without `FAIL_MODE` during that wait.

```
uv run --env-file .env python - <<'EOF'
import asyncio, httpx
from services.order_service.app.infrastructure.payment_client import
        PaymentClient

async def main():
    async with httpx.AsyncClient() as http:
        client = PaymentClient("http://localhost:8003", http)
        for i in range(5):
            try:
                await client.authorize(f"o-{i}", 10)
            except Exception:
                pass  # expected: PaymentUnavailable while
        FAIL_MODE=500
        print("Restart the payment mock WITHOUT FAIL_MODE within 40
        seconds...")
        await asyncio.sleep(40)  # longer than timeout_duration (30
        seconds)
        print("probe:", await client.authorize("o-probe", 10))  #
        half-open probe succeeds
        print("after:", await client.authorize("o-after", 10))  #
        circuit is closed again
asyncio.run(main())
EOF
# expect a probe result and an "after" result, not CircuitBreakerError
```

Step 3.2: use a bulkhead for a fragile dependency

Problem: when payment slows down, every incoming request waits for it. Even with timeouts, a traffic spike can start thousands of payment calls on one replica and exhaust its connections and memory.

Pattern: a bulkhead (a semaphore) caps how many payment calls one replica runs at once. Extra requests either wait briefly or receive `503` right away, so the failure stays inside the payment path.

```
# services/order_service/app/application/authorize_payment.py
# Step 3.2: bulkhead semaphore around the payment call.
import asyncio
from aiobreaker import CircuitBreakerError
```

```
from ..api.errors import DomainError
from ..infrastructure.payment_client import PaymentDeclined,
        PaymentUnavailable

payment_slots = asyncio.Semaphore(20)

async def authorize_with_bulkhead(client, order_id, total):
    try:
        async with asyncio.timeout(1.2):
            async with payment_slots:
                return await client.authorize(order_id, total)
    except TimeoutError:
        raise DomainError("PAYMENT_UNAVAILABLE", "Try again later",
        503)
    except (PaymentUnavailable, CircuitBreakerError):
        # CircuitBreakerError means the breaker is open: payment is
        treated as unavailable without a network call.
        raise DomainError("PAYMENT_UNAVAILABLE", "Try again later",
        503)
    except PaymentDeclined:
        raise DomainError("PAYMENT_DECLINED", "Payment was declined",
        402)
```

The semaphore is a **local bulkhead**, not a distributed rate limit. It prevents 1,000 incoming requests from creating 1,000 in-flight payment calls per replica. Keep the limit observable and tune it against payment capacity.

Verify the bulkhead and the error mapping: create `tests/test_authorize_payment.py`.

```
# tests/test_authorize_payment.py
import asyncio

import pytest

from services.order_service.app.api.errors import DomainError
from services.order_service.app.application.authorize_payment import
        authorize_with_bulkhead
from services.order_service.app.infrastructure.payment_client import
        PaymentUnavailable


class SlowClient:
    def __init__(self):
        self.in_flight = 0
        self.peak = 0

    async def authorize(self, order_id, total):
        self.in_flight += 1
        self.peak = max(self.peak, self.in_flight)
        await asyncio.sleep(0.05)
        self.in_flight -= 1
        return "auth-1"


class UnavailableClient:
    async def authorize(self, order_id, total):
        raise PaymentUnavailable("payment down")


@pytest.mark.asyncio
async def test_bulkhead_caps_concurrent_payment_calls():
    client = SlowClient()
```

```
    await asyncio.gather(*(authorize_with_bulkhead(client, f"o-{i}",
        10) for i in range(100)))
    assert client.peak <= 20


@pytest.mark.asyncio
async def test_unavailable_payment_maps_to_503():
    with pytest.raises(DomainError) as exc:
        await authorize_with_bulkhead(UnavailableClient(), "o-1", 10)
    assert exc.value.status_code == 503

uv run pytest tests/test_authorize_payment.py -v
# expect both tests PASSED: peak concurrency never exceeds 20, and
        PaymentUnavailable becomes a 503
```

Step 3.3: make failures visible

Problem: when a call fails, the caller sees a generic 500 or a raw provider error. On-call engineers cannot tell a timeout from an open breaker from a full bulkhead.

Pattern: map each dependency failure to a stable `503` with a retry hint, hide provider details, and emit metrics for the signals that explain the failure: duration, breaker state, timeouts, and bulkhead occupancy.

Return `503` plus a safe retry hint when the request cannot proceed. Do not reveal payment provider errors. Emit metrics: request duration, breaker state, downstream timeout count, and in-flight bulkhead slots.

Verify: the `test_unavailable_payment_maps_to_503` test above checks the status code. Through the shared handler from Lab 1, the body is `{"code": "PAYMENT_UNAVAILABLE", "message": "Try again later", "trace_id": "..."}`, with no provider text.

Failure clinic

Bad "fix"	Why it makes the incident worse	Replace it with
retry every exception three times	load triples while a dependency is sick	retry only transport/5xx failures, with jitter and a global deadline
retry `POST /charge` without a key	a timeout may mean the provider already charged	downstream idempotency key based on your order/operation ID
circuit breaker around all errors	bad request/declined card opens breaker	count only dependency availability failures
unbounded `gather()` over 10,000 operations	exhausts connections/memory	bounded concurrency and queues
`except Exception: return {"success": false}`	loses failure classification and observability	map expected exceptions, log unexpected exceptions with trace ID

Exercise: inject `await asyncio.sleep(3)` in payment. Send 50 concurrent order requests. Observe timeout, semaphore limiting, then circuit opening. Remove the delay and observe half-open recovery.

Lab 4 — Let the platform route traffic: gateway and discovery

Problem: the browser/AI client calls every service directly, carries internal addresses, and each service repeats auth, CORS, and rate limiting differently.

Step 4.1: define the gateway boundary — local Compose implementation

Problem: clients call each service directly, carry internal addresses, and each service repeats authentication, CORS, and rate limiting in its own way.

Pattern: one gateway at the edge owns the public API: routing, authentication, rate limits, and request limits. The business logic stays in the services.

Environment for this lab: Docker Compose only. You are moving from Labs 0-3, where the FastAPI process ran on your host and Postgres/Redpanda ran in containers, to a local multi-container system. Kubernetes appears for the first time in Lab 8.

The API gateway is the north-south edge. It performs API authentication, coarse rate limits, request-size limits, routing/versioning, TLS termination, and audit correlation. It should **not** contain order business rules or become a database-backed monolith.

The local request path is deliberately simple:

```
%% Local request path: client to Kong to order-service.
sequenceDiagram
  participant C as curl / AI client
  participant G as Kong localhost:8000
  participant O as order-service:8000
  C->>G: POST /api/v1/orders
  G->>O: POST /v1/orders
  O-->>G: 202 + order_id
  G-->>C: 202 + order_id
```

Step 4.1a: add the container image used from this point onward

Problem: the laptop, Compose, and Kubernetes can each run different dependency versions, so "it passed locally" proves little.

Pattern: build one immutable image from the locked dependencies and run that same image everywhere.

Create `services/order_service/Dockerfile`. Lab 8 reuses this same immutable image; it does not introduce a second packaging approach.

```
# Container image for the order service, reused unchanged through Lab
        8.
FROM python:3.12-slim
WORKDIR /app
RUN useradd --system --create-home appuser

COPY pyproject.toml uv.lock ./
RUN pip install --no-cache-dir uv && uv sync --frozen --no-dev

COPY services ./services
USER appuser
EXPOSE 8000
CMD ["uv", "run", "uvicorn", "services.order_service.app.main:app", "-
        -host", "0.0.0.0", "--port", "8000"]
```

Verify the image: build it and import the application inside the container.

```
docker build -t order-service:workshop -f
        services/order_service/Dockerfile .
docker run --rm order-service:workshop uv run python -c "from
        services.order_service.app.main import app; print(app.title)"
# expect: Order Service
```

Step 4.1b: run Kong and the service in Compose

Problem: the application runs on the host, so nothing has one stable entry point, and each dependency needs a different address on each machine.

Pattern: put every service on the Compose network, where they reach each other by service name. Kong is the only service with a host port.

Create `infra/.env.compose`. Its hostnames are Compose service names, not laptop hostnames:

```
# Compose-network hostnames for containers (Lab 4+).
DATABASE_URL=postgresql+asyncpg://orders:orders@postgres:5432/orders
INVENTORY_URL=http://inventory-service:8000
PAYMENT_URL=http://payment-service:8000
KAFKA_BOOTSTRAP_SERVERS=redpanda:9092
```

Add these services to `infra/compose.yaml` below the existing Postgres and Redpanda services:

```
# order-service, inventory/payment mocks, and Kong added to
        infra/compose.yaml.
  order-service:
    build:
      context: ..
      dockerfile: services/order_service/Dockerfile
    env_file: .env.compose
    expose: ["8000"]                # internal only; no direct laptop
        port
    depends_on:
      postgres:
        condition: service_healthy

  inventory-service:
    build:
      context: ..
      dockerfile: services/order_service/Dockerfile
    command: ["uv", "run", "uvicorn",
        "services.mock_dependency.main:app", "--host", "0.0.0.0", "--
        port", "8000"]
    environment:
      DEPENDENCY_NAME: inventory
    expose: ["8000"]

  payment-service:
    build:
      context: ..
      dockerfile: services/order_service/Dockerfile
    command: ["uv", "run", "uvicorn",
        "services.mock_dependency.main:app", "--host", "0.0.0.0", "--
        port", "8000"]
    environment:
      DEPENDENCY_NAME: payment
    expose: ["8000"]

  kong:
    image: kong:3.7
    environment:
```

```
    KONG_DATABASE: "off"
    KONG_DECLARATIVE_CONFIG: /kong/kong.yml
    KONG_PROXY_LISTEN: 0.0.0.0:8000
    KONG_ADMIN_LISTEN: 0.0.0.0:8001
  volumes:
    - ./gateway/kong.yml:/kong/kong.yml:ro
  ports:
    - "8000:8000"                  # client entry point
    - "8001:8001"                  # local inspection only; never
      expose in production
  depends_on:
    - order-service
```

> [!IMPORTANT] The `order-service` uses `expose`, not `ports`. From this lab onward, the laptop client must enter through Kong. The only exception is temporary debugging with `docker compose exec`.

Step 4.1c: configure Kong with a path that actually matches FastAPI

Problem: clients call `/api/v1/orders`, but FastAPI serves `/v1/orders`. Rate limits and payload limits are also needed at the edge, not repeated in every service.

Pattern: the gateway owns the public path, strips the prefix before forwarding, and applies coarse rate and size limits.

Create `infra/gateway/kong.yml`:

```
# Kong routes /api to order-service with rate and request-size limits.
_format_version: "3.0"
services:
  - name: order-service
    url: http://order-service:8000
    routes:
      - name: order-api
        paths: ["/api"]
        strip_path: true
    plugins:
      - name: rate-limiting
        config:
          minute: 60
          policy: local              # local-learning setting; Lab 9
        uses shared Redis policy
      - name: request-size-limiting
        config:
          allowed_payload_size: 1
```

`strip_path: true` is essential here. It maps the external `/api/v1/orders` route to FastAPI's internal `/v1/orders` route. The earlier `strip_path: false` example would have forwarded `/api/v1/orders` and resulted in a FastAPI 404.

Verify the gateway: the internal path must be unreachable through Kong, and the rate limit must trigger.

```
# expect 404: Kong has no route for /v1/orders
curl -s -o /dev/null -w "%{http_code}\n" localhost:8000/v1/orders

# expect a count with some 429 lines: 61 requests in one minute exceed
        the 60-per-minute limit
for i in $(seq 1 61); do curl -s -o /dev/null -w "%{http_code}\n"
        localhost:8000/api/livez; done | sort | uniq -c
```

Step 4.1d: start and verify the boundary

Problem: a configuration can look right in files and still fail when started together.

Pattern: start the full Compose stack once, then test the public path and the blocked internal path from the host.

```
# Start the stack and verify the Kong /api boundary.
docker compose -f infra/compose.yaml up --build -d
docker compose -f infra/compose.yaml ps
curl -i http://localhost:8000/api/livez
curl -i -X POST http://localhost:8000/api/v1/orders \
  -H 'content-type: application/json' \
  -H 'Idempotency-Key: order-demo-key-0001' \
  -d '{"customer_id":"11111111-1111-1111-1111-111111111111","lines":
        [{"sku":"SKU-001","quantity":1,"unit_price":"49.90"}]}'
```

Expected result: Kong returns `202` and an order ID. If you call `http://localhost:8000/v1/orders` directly, it should fail because that port belongs to Kong, not the application.

In the service, trust forwarded identity headers only when requests can come exclusively through a gateway/mesh. In production, enforce that with network policy and signed JWT validation; otherwise any caller can forge `X-User-Id`.

Step 4.2: service discovery — Compose now, Kubernetes later

Problem: hard-coded IP addresses change whenever a container or Pod restarts, so calls to a dependency fail after a redeploy.

Pattern: call a stable name and let DNS resolve it to a healthy instance. The name is the contract; the address is an implementation detail.

Environment	This course uses it in	How `order-service` finds inventory	What not to do
Docker Compose	**This lab and local labs**	DNS name `http://inventory-service:8000`	hard-code laptop IPs
Kubernetes	**Lab 8 onward**	`http://inventory-service.orders.svc.cluster.local` resolved by CoreDNS	inject individual Pod IPs into configuration
Mesh	**Lab 8 onward**	call the Kubernetes service name; sidecar load-balances healthy endpoints	build a Python registry client alongside the platform registry

For the Compose lab, service names in `compose.yaml` are DNS names on Compose's default network. For Kubernetes, a `Service` gives a stable virtual address while Pods are created and removed; readiness gates which Pods become endpoints. The concept is the same, but the platform implementation differs.

Verify discovery: from inside the `order-service` container, resolve the inventory name and call its health endpoint.

```
docker compose -f infra/compose.yaml exec -T order-service uv run
        python -c "
import httpx
print(httpx.get('http://inventory-service:8000/health').json())
"
# expect {'dependency': 'inventory', 'status': 'ok'}
```

▶ **Why not use Kubernetes discovery in Lab 4?**

Failure clinic

Symptom	Cause	Fix
works locally but returns `Name or service not known` in cluster	Compose service DNS name copied into a different namespace	use fully-qualified service address or correct namespace
service routes to a Pod still starting	liveness exists but readiness does not represent dependency initialization	configure readiness and only return ready when the service can perform its required startup work
gateway says `429`, service dashboards do not show it	request is rejected before FastAPI	monitor gateway metrics/logs as a first-class service
spoofed identity header	service is reachable outside the trusted ingress	validate JWT in service or restrict network paths and use mesh identity

Lab 5 — Decouple the workflow with async messaging and a transactional outbox

Problem: `POST /orders` creates an order in Postgres and then publishes `OrderCreated` to Kafka. If the process dies between those steps, the order exists but shipping/notification never learns about it. If it publishes first and DB fails, consumers act on an order that does not exist.

Step 5.1: write the event in the same transaction

Problem: the order is committed to Postgres and then published to Kafka. If the process dies between the two, the order exists but no one hears about it. If the publish happens first and the commit fails, consumers act on an order that does not exist.

Pattern: write the event to an `outbox` table in the same transaction as the order. Both commit or both roll back. A separate publisher then relays unpublished rows to Kafka.

```
-- Outbox table written in the same transaction as the order (Lab 5).
CREATE TABLE outbox (
  id UUID PRIMARY KEY,
  aggregate_id UUID NOT NULL,
  event_type TEXT NOT NULL,
  payload JSONB NOT NULL,
  occurred_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  published_at TIMESTAMPTZ
);
CREATE INDEX outbox_unpublished_idx ON outbox (occurred_at) WHERE
        published_at IS NULL;
```

Inside the *same* transaction that inserts the order, allocate the event ID once and persist the complete event envelope:

```
# Write the OrderCreated event in the same transaction as the order.
event_id = uuid4()
await session.execute(text("""
```

```
  INSERT INTO outbox(id, aggregate_id, event_type, payload)
  VALUES (:id, :order_id, 'OrderCreated', CAST(:payload AS jsonb))
"""), {"id": event_id, "order_id": order_id,
           "payload": dumps({"event_id": str(event_id), "order_id":
        str(order_id),
                                "customer_id": str(command.customer_id),
        "version": 1})})
```

Now a publisher process may safely retry delivery:

```
# services/order_service/app/infrastructure/outbox_publisher.py — use
        SKIP LOCKED so replicas do not publish the same row
        concurrently
# Publisher: send unpublished outbox rows to Kafka and mark them
        published.
from json import dumps
from sqlalchemy import text

async def publish_batch(session, producer):
    async with session.begin():
        rows = (await session.execute(text("""
          SELECT id, aggregate_id, event_type, payload FROM outbox
          WHERE published_at IS NULL
          ORDER BY occurred_at FOR UPDATE SKIP LOCKED LIMIT 100
        """))).mappings().all()
        for row in rows:
            body = dumps(row["payload"]).encode()
            await producer.send_and_wait(
                "orders.events",
                body,
                key=str(row["aggregate_id"]).encode(),
            )
            await session.execute(
                text("UPDATE outbox SET published_at = now() WHERE id
        = :id"),
                {"id": row["id"]},
            )
```

Use the aggregate ID, not the event ID, as the Kafka key so all events for one order stay in partition order. The publisher example uses a short database transaction and small batch for learning. For high-volume systems, use a lease/claim strategy or CDC so database locks are not held during broker calls.

For high volume, use CDC (Debezium) to publish the outbox table. The transaction principle remains unchanged.

Verify atomicity: temporarily add `raise RuntimeError("simulated crash")` directly after the `outbox` insert in `create_order`, send a valid order, and expect a `500`. Then compare counts before and after:

```
docker compose -f infra/compose.yaml exec -T postgres psql -U orders -
        d orders \
  -c "select (select count(*) from orders) as orders, (select count(*)
        from outbox) as outbox;"
# both counts must be unchanged after the failed request. Remove the
        raise afterward.
```

Verify publishing: after a normal request, the outbox row has `published_at IS NULL`. Run the publisher once, and the row must be marked published.

```
uv run --env-file .env python - <<'EOF'
import asyncio
from aiokafka import AIOKafkaProducer
from services.order_service.app.infrastructure.db import
        SessionFactory
```

```
from services.order_service.app.infrastructure.outbox_publisher import
        publish_batch

async def main():
    producer = AIOKafkaProducer(bootstrap_servers="localhost:9092")
    await producer.start()
    try:
        async with SessionFactory() as session:
            await publish_batch(session, producer)
    finally:
        await producer.stop()

asyncio.run(main())
EOF
docker compose -f infra/compose.yaml exec -T postgres psql -U orders -
        d orders -c "select count(*) from outbox where published_at is
        null;"
# expect 0
```

Step 5.2: accept at-least-once delivery

Problem: the publisher can crash after Kafka accepts an event but before it marks the row published. The event is then sent again, so a consumer can process the same order twice.

Pattern: the consumer records each processed event ID under a unique key in the same transaction as its work. A duplicate hits the conflict and does nothing.

Kafka producer acknowledgement plus a database update is still not one global atomic transaction. A crash after Kafka accepts an event but before `published_at` commits means the event can be sent twice. Therefore consumers must be idempotent.

Verify the dedupe key: create the `processed_events` table from the SQL above, then run the same insert twice. The first returns one row; the second returns none.

```
docker compose -f infra/compose.yaml exec -T postgres psql -U orders -
        d orders -c \
  "INSERT INTO processed_events(consumer_name, event_id) VALUES
        ('shipping', '8bd1f4c2-0000-4000-8000-000000000001') ON
        CONFLICT DO NOTHING RETURNING event_id;"
# first run: INSERT 0 1 with the event_id row
# second run: INSERT 0 0

-- Consumer dedupe table keyed by consumer name and event ID.
CREATE TABLE processed_events (
  consumer_name TEXT NOT NULL,
  event_id UUID NOT NULL,
  processed_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (consumer_name, event_id)
);

# Idempotent consumer: skip events already recorded in
        processed_events.
async def consume_shipping(event, session):
    async with session.begin():
        seen = await session.execute(text("""
          INSERT INTO processed_events(consumer_name, event_id)
          VALUES ('shipping', :event_id) ON CONFLICT DO NOTHING
          RETURNING event_id
        """), {"event_id": event["event_id"]})
        if seen.scalar_one_or_none() is None:
            return  # duplicate: acknowledge safely
        # Create shipment with an order_id unique constraint.
```

Failure clinic

Symptom	Why	Fix
event missing after a restart	DB commit and publish were two independent actions	transactional outbox + publisher/CDC
customer receives two confirmations	delivery is at least once	inbox/dedupe table and idempotent business operation
one poison event blocks a partition forever	consumer retries an invalid schema/event infinitely	bounded retry topics, dead-letter topic, alert and repair runbook
events arrive out of order for an order	keys were random / topic partitions reorder globally	use `order_id` as Kafka message key; version the aggregate
consumers break after producer deployment	event was an undocumented internal JSON blob	use versioned event envelopes and compatibility rules; never rename/delete required fields casually

Lab 6 — Coordinate distributed work with a saga

Problem: "create order → reserve inventory → authorize payment → create shipment" is not a single database transaction. Two-phase commit across services is slow, fragile, and rarely appropriate. We need a business-level sequence with explicit compensations.

Step 6.1: model the saga states

Problem: reserve stock, authorize payment, and create a shipment are three separate services. If shipping fails after payment, no single database transaction can undo the earlier steps.

Pattern: an orchestrated saga. One component stores the order's state durably, moves it forward one step at a time, and runs a compensating action for each completed step when a later step fails.

```
%% Saga states and the compensation transitions for each failure.
stateDiagram-v2
  [*] --> PENDING
  PENDING --> STOCK_RESERVED: reserve succeeds
  STOCK_RESERVED --> PAYMENT_AUTHORIZED: payment succeeds
  PAYMENT_AUTHORIZED --> CONFIRMED: shipment created
  STOCK_RESERVED --> CANCELLED: payment fails / release stock
  PAYMENT_AUTHORIZED --> CANCELLED: shipment fails / void payment,
release stock
```

Store saga state durably in the Order database. The workflow may be triggered by events; it must survive restarts and be safe to retry.

```python
# Orchestrated saga step: advance an order using durable state and
        compensations.
async def advance_order(order_id, inventory, payment, shipping, repo):
    order = await repo.lock_for_update(order_id)
    if order.status == "PENDING":
```

```
        order.reservation_id = await inventory.reserve(order_id,
        order.lines)
        order.status = "STOCK_RESERVED"
        await repo.save(order)

    if order.status == "STOCK_RESERVED":
        try:
            order.authorization_id = await payment.authorize(order_id,
        order.total)
            order.status = "PAYMENT_AUTHORIZED"
            await repo.save(order)
        except PaymentDeclined:
            await inventory.release(order.reservation_id)  #
        compensation, itself idempotent
            order.status = "CANCELLED"
            await repo.save(order)
            return

    if order.status == "PAYMENT_AUTHORIZED":
        try:
            order.shipment_id = await shipping.create(order_id,
        order.lines)
            order.status = "CONFIRMED"
        except ShippingUnavailable:
            await payment.void(order.authorization_id)
            await inventory.release(order.reservation_id)
            order.status = "CANCELLED"
        await repo.save(order)
```

This is an **orchestrated saga**: one component owns the state machine. Choreography—each service reacts to events—works for simple flows, but becomes difficult to reason about when the compensation path spans many services.

Step 6.2: distinguish a compensation from rollback

Problem: teams often assume a failed step can be rolled back. Stock release and payment capture happen in other systems, so the "undo" is a new business event, not an erased row.

Pattern: design each compensation before its forward step, and decide which actions are irreversible and what happens then.

`release inventory` cannot erase history. It creates a new business fact. A payment authorization can be voided; a settled payment needs a refund. Design compensations before the forward action and document the "irreversible" boundary.

Failure clinic

Scenario	Correct response
reserve request timed out; inventory may have reserved	retry using the same operation ID, then query by operation ID before assuming failure
order coordinator crashes after payment authorization	replay saga from durable state; make `authorize(order_id)` idempotent
release stock fails after a declined card	mark order `COMPENSATION_PENDING`, retry asynchronously, page on age/SLO; do not lie that it is cleanly cancelled
shipment created but confirmation response was lost	lookup shipment by `order_id` rather than create another shipment
payment has settled	use refund policy instead of void; require a human/business rule if refund cannot be

Scenario	Correct response
	automatic

Exercise: make shipping return 503 after payment success. Show the order entering `COMPENSATION_PENDING`, then recover it after shipping/payment services come back.

Verify the compensation path: save the `advance_order` function from Step 6.1 as `services/order_service/app/application/order_saga.py`, and define `ShippingUnavailable` and `PaymentDeclined` in that module. Then create `tests/test_order_saga.py`:

```
# tests/test_order_saga.py
from types import SimpleNamespace

import pytest

from services.order_service.app.application.order_saga import
        ShippingUnavailable, advance_order


class FakeRepo:
    def __init__(self, order):
        self.order = order

    async def lock_for_update(self, order_id):
        return self.order

    async def save(self, order):
        pass


class FakeInventory:
    def __init__(self):
        self.released = []

    async def reserve(self, order_id, lines):
        return "res-1"

    async def release(self, reservation_id):
        self.released.append(reservation_id)


class FakePayment:
    def __init__(self):
        self.voided = []

    async def authorize(self, order_id, total):
        return "auth-1"

    async def void(self, authorization_id):
        self.voided.append(authorization_id)


class FailingShipping:
    async def create(self, order_id, lines):
        raise ShippingUnavailable()


@pytest.mark.asyncio
async def test_shipping_failure_voids_payment_and_releases_stock():
    order = SimpleNamespace(status="PENDING", lines=[], total=10,
```

```
                            reservation_id=None,
        authorization_id=None, shipment_id=None)
    inventory, payment = FakeInventory(), FakePayment()

    await advance_order("o-1", inventory, payment, FailingShipping(),
        FakeRepo(order))

    assert order.status == "CANCELLED"
    assert payment.voided == ["auth-1"]
    assert inventory.released == ["res-1"]

uv run pytest tests/test_order_saga.py -v
# expect PASSED: the failed shipment leaves no authorization held and
        no stock reserved
```

Lab 7 — Use CQRS and event sourcing for the right problem

Problem: the transactional `orders` table handles command consistency well, but support dashboards need a denormalized, filterable timeline: "all orders delayed after payment authorization." Complex reports now hurt command latency and every table change breaks read views.

Step 7.1: apply CQRS before event sourcing

Problem: support dashboards need filtered, denormalized order timelines. Running those reports on the `orders` table slows order writes, and every schema change breaks the reports.

Pattern: separate the write model from a read model. A projection consumer builds the read model from events, so queries never touch the command path.

CQRS means **separate write and read models**, not necessarily separate databases or microservices. The command side protects invariants. The query side is optimized for questions.

```
-- Read model for support dashboards, updated by projection consumers
        (Lab 7).
CREATE TABLE order_summary (
  order_id UUID PRIMARY KEY,
  customer_id UUID NOT NULL,
  status TEXT NOT NULL,
  total NUMERIC(10,2) NOT NULL,
  payment_status TEXT,
  shipment_status TEXT,
  updated_at TIMESTAMPTZ NOT NULL
);
```

A projection consumer listens to versioned order events and upserts `order_summary`. `GET /v1/orders/{id}` can read this model. Tell callers its data is **eventually consistent**—a just-accepted order might not appear in a dashboard for seconds.

Verify the projection is idempotent: apply the same `OrderCreated` event twice, as a redelivery would. Expect one summary row. Apply `order_summary` from Step 7.1 first.

```
uv run --env-file .env python - <<'EOF'
import asyncio, uuid
from sqlalchemy import text
```

```
from services.order_service.app.infrastructure.db import
        SessionFactory
from services.order_service.app.application.order_projection import
        apply_event

async def main():
    oid, cid = str(uuid.uuid4()), str(uuid.uuid4())
    event = {"type": "OrderCreated", "data": {"order_id": oid,
        "customer_id": cid, "total": "49.90"}}
    async with SessionFactory() as session:
        async with session.begin():
            await apply_event(event, session)
            await apply_event(event, session)  # duplicate delivery
    async with SessionFactory() as session:
        rows = (await session.execute(
            text("select count(*) from order_summary where order_id =
        :id"), {"id": oid})).scalar()
        print("summary rows:", rows)  # expect 1

asyncio.run(main())
EOF

# Projection handler: apply order events to the order_summary read
        model.
async def apply_event(event, session):
    # Do dedupe/version checks first (see Lab 5).
    if event["type"] == "OrderCreated":
        await session.execute(text("""
          INSERT INTO order_summary(order_id, customer_id, status,
        total, updated_at)
          VALUES (:id, :customer_id, 'PENDING', :total, now())
          ON CONFLICT (order_id) DO NOTHING
        """), event["data"])
    elif event["type"] == "PaymentAuthorized":
        await session.execute(text("UPDATE order_summary SET
        payment_status='AUTHORIZED', updated_at=now() WHERE
        order_id=:id"), event["data"])
```

Step 7.2: event sourcing is a stronger choice

Problem: two commands read the same order and both append a change, so one silently overwrites the other.

Pattern: store the event stream as the source of truth and enforce an expected version on every append. A stale writer fails on the primary key instead of overwriting.

In event sourcing, the event stream is the source of truth; current order state is rebuilt by applying events.

```
-- Event store: append-only events per order, keyed by version (Lab
        7).
CREATE TABLE order_events (
  order_id UUID NOT NULL,
  version INTEGER NOT NULL,
  event_id UUID NOT NULL UNIQUE,
  event_type TEXT NOT NULL,
  data JSONB NOT NULL,
  occurred_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (order_id, version)
);
```

Append with optimistic concurrency:

```
-- Optimistic-concurrency append: fails if the expected version is
        stale.
INSERT INTO order_events(order_id, version, event_id, event_type,
        data)
VALUES (:order_id, :expected_version + 1, :event_id, 'StockReserved',
        :data);
```

If another command already appended version 5, the primary-key collision tells you your state was stale. Reload events, re-evaluate the command, and either retry or reject.

Verify optimistic concurrency: append version 1 twice. The first insert succeeds; the second must fail on the primary key.

```
docker compose -f infra/compose.yaml exec -T postgres psql -U orders -
        d orders <<'SQL'
INSERT INTO order_events(order_id, version, event_id, event_type,
        data)
VALUES ('11111111-2222-3333-4444-555555555555', 1, gen_random_uuid(),
        'StockReserved', '{}');
INSERT INTO order_events(order_id, version, event_id, event_type,
        data)
VALUES ('11111111-2222-3333-4444-555555555555', 1, gen_random_uuid(),
        'StockReserved', '{}');
SQL
# second insert: ERROR: duplicate key value violates unique constraint
        "order_events_pkey"
```

Do not use event sourcing merely because it sounds advanced. It earns its complexity when immutable audit history, temporal reconstruction, and multiple independent projections are central product needs (financial ledger, compliance, complex order lifecycle). For many services, the Lab 2 transactional model plus Lab 5 outbox is the better default.

Failure clinic

Symptom	Cause	Fix
dashboard says PENDING after order is confirmed	projection lag or failed consumer	expose last-applied offset/age; replay idempotent projection; set freshness SLO
replay changes the result after code deployment	event handler logic is not versioned/tested	keep events immutable, add upcasters/versions, test historical replay
two commands race and overwrite an order	no expected version check	optimistic concurrency on aggregate version
event store grows and rehydration is slow	loading full history per command	snapshots are a cache; validate snapshot version then apply newer events

Lab 8 — Production platform: containers, Kubernetes, sidecars, service mesh

Problem: each service must implement mTLS, tracing, retries, traffic shifting, and network policy perfectly. That creates inconsistent reliability and security code in every repository.

Step 8.0: create the optional local Kubernetes platform

Problem: the same service runs differently in Compose and in a cluster, so platform concerns such as probes, sidecars, and routing are untested until production.

Pattern: a local cluster with Istio, so the platform behavior can be checked before a production rollout.

Complete this once, only after Labs 0-7 work in Compose. If you use Minikube, translate the `kind` image-loading command in Step 8.1; the Kubernetes manifests remain the same.

```
# Create a local kind cluster and confirm Istio is running.
kind create cluster --name workshop
istioctl install --set profile=demo -y
kubectl get nodes
kubectl get pods -n istio-system
```

Expected result: the cluster has one Ready control-plane node and the Istio control-plane Pods are Running. If `istioctl` is unavailable, install it from the official Istio release matching the version you intend to use; do not continue with the mesh exercises without a working control plane.

Verify the control plane:

```
istioctl verify-install   # expect no errors
```

Step 8.1: promote the same image built in Lab 4

Problem: a separate image for the cluster means the tested artifact is not the deployed artifact.

Pattern: promote one image by identity. Build it once, load or push that exact build, and reference it by a version or digest in every environment.

Lab 4 already created the **services/order_service/Dockerfile** so learners do not maintain separate "Compose" and "Kubernetes" images. Build the identical source once, then deploy its immutable digest to each environment.

```
# Build the image once and load it into the local cluster.
docker build -t order-service:workshop -f
        services/order_service/Dockerfile .

# Choose exactly one local-cluster loading command.
kind load docker-image order-service:workshop --name workshop
# minikube image load order-service:workshop
```

Verify the image is in the cluster (kind example; the node name is `<cluster>-control-plane`):

```
docker exec workshop-control-plane crictl images | grep order-service
# expect an order-service:workshop row
```

For a real registry, replace **order-service:workshop** with a versioned image digest. Inject configuration/secrets at runtime. Never bake environment files, API keys, or database URLs into the image.

Step 8.2: deploy statelessly

Problem: a service that keeps state in memory or depends on Pod identity cannot survive restarts, rollouts, or autoscaling. Its health checks must also match what it actually needs to serve traffic.

Pattern: keep the Pod stateless, configure it through a Secret, and split probes: `startupProbe` covers slow startup, `livenessProbe` restarts a dead process, and `readinessProbe` controls traffic.

Create the namespace and inject the sidecar **before** creating the Deployment:

```
# Create the namespace and enable sidecar injection before the
        Deployment, then store secrets.
kubectl create namespace orders
kubectl label namespace orders istio-injection=enabled
kubectl -n orders create secret generic order-service-secrets \
  --from-
        literal=DATABASE_URL='postgresql+asyncpg://placeholder:placeholder@placeholder:543
        \
  --from-literal=INVENTORY_URL='http://inventory-service' \
  --from-literal=PAYMENT_URL='http://payment-service' \
  --from-literal=KAFKA_BOOTSTRAP_SERVERS='placeholder:9092'
```

> [!NOTE] This Lab 8 cluster exercise verifies Pod lifecycle, discovery, sidecar injection, and traffic policy. It does **not** move Postgres/Redpanda into Kubernetes; keep the stateful Labs 2-7 local in Compose. The placeholder URLs are safe because the current health endpoints do not call a dependency. Do not use this placeholder configuration for a real workload.

```
# k8s/base/order-service.yaml
# Deployment, Service, and HPA for order-service with probes and
        secrets.
apiVersion: apps/v1
kind: Deployment
metadata: { name: order-service }
spec:
  replicas: 3
  selector: { matchLabels: { app: order-service } }
  template:
    metadata: { labels: { app: order-service } }
    spec:
      containers:
        - name: app
          image: order-service:workshop
          imagePullPolicy: IfNotPresent
          ports: [{ containerPort: 8000 }]
          envFrom: [{ secretRef: { name: order-service-secrets } }]
          resources:
            requests: { cpu: "250m", memory: "512Mi" }
            limits: { cpu: "1", memory: "1Gi" }
          startupProbe:
            httpGet: { path: /livez, port: 8000 }
            failureThreshold: 30
            periodSeconds: 2
          livenessProbe:
            httpGet: { path: /livez, port: 8000 }
          readinessProbe:
            httpGet: { path: /readyz, port: 8000 }
---
apiVersion: v1
kind: Service
metadata: { name: order-service }
spec:
  selector: { app: order-service }
  ports: [{ port: 80, targetPort: 8000 }]
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: order-service }
```

```
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: order-service
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70

# Deploy to the orders namespace and check sidecar, readiness, and
        liveness.
kubectl -n orders apply -f k8s/base/order-service.yaml
kubectl -n orders get pods,svc,hpa
kubectl -n orders get pod -l app=order-service -o
        jsonpath='{.items[0].spec.containers[*].name}'
kubectl -n orders port-forward service/order-service 8080:80
# In another terminal:
curl -i http://localhost:8080/livez
```

Expected result: the container list includes **app istio-proxy**, at least two Pods become ready, and **/livez** returns **200**. If **istio-proxy** is absent, stop and verify that Istio is installed and the namespace label existed before Pod creation.

Verify the rollout and probes: the Deployment must become ready, and a Pod must receive traffic only when ready.

```
kubectl -n orders rollout status deployment/order-service   # expect:
        successfully rolled out
kubectl -n orders get pods -l app=order-service              # expect
        READY 2/2 for each Pod
```

The HPA is included to show the resource-scaling contract. Run `kubectl -n orders get hpa`. If CPU shows `<unknown>`, your local cluster has no Metrics Server; install/enable it before trying to validate CPU scaling. Lab 9 explains why queue lag and concurrency limits are often better scaling signals for asynchronous workers.

Step 8.3: understand the sidecar pattern

Problem: every service re-implements mTLS, retries, and telemetry, and each implementation drifts.

Pattern: a sidecar proxy beside each Pod handles those concerns uniformly. The application keeps its business logic.

A **sidecar** is a helper container running beside the application in the same Pod. It shares lifecycle/network namespace but owns a cross-cutting concern. Examples: Envoy proxy, log shipper, secret agent.

The service mesh injects an Envoy sidecar. Calls from FastAPI to `inventory-service` are transparently intercepted; Envoy can enforce mTLS, collect request metrics/traces, apply connection pooling, and load-balance healthy endpoints.

Important boundary: do not apply retry both in FastAPI *and* in the mesh without a single retry budget. Two layers of three retries can create nine requests; add a gateway retry and it can become 27.

Verify the sidecar is attached and synced:

```
istioctl proxy-status | grep order-service   # expect SYNCED for each
        order-service Pod
```

Step 8.4: translate the local gateway path to an Istio ingress gateway

Problem: the Kong gateway from Lab 4 does not exist in the cluster, so the same external path must be recreated with the platform's own gateway.

Pattern: an Istio ingress `Gateway` plus `VirtualService` rewrites `/api` to the service's paths, matching the Lab 4 contract.

Kong is the local Compose gateway in Lab 4. In this Kubernetes lab, use Istio's ingress gateway so you can observe one platform-owned north-south entry point alongside the sidecar mesh. In a real platform, choose the organization-standard gateway; do not expose both Kong and Istio gateway for the same API path without a clear reason.

```
# k8s/istio/order-ingress.yaml
# Istio ingress: expose /api through the platform gateway (Lab 8).
apiVersion: networking.istio.io/v1
kind: Gateway
metadata: { name: orders-gateway }
spec:
  selector:
    istio: ingressgateway
  servers:
    - port: { number: 80, name: http, protocol: HTTP }
      hosts: ["*"]
---
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata: { name: order-service-edge }
spec:
  hosts: ["*"]
  gateways: [orders-gateway]
  http:
    - match:
        - uri: { prefix: "/api" }
      rewrite:
        uriRegexRewrite:
          match: "^/api/(.*)"
          rewrite: "/\\1"
      route:
        - destination:
            host: order-service.orders.svc.cluster.local
            port: { number: 80 }

# Apply the ingress and verify the /api rewrite through Istio.
kubectl -n orders apply -f k8s/istio/order-ingress.yaml
kubectl -n istio-system port-forward service/istio-ingressgateway
        8080:80
# In another terminal:
curl -i http://localhost:8080/api/livez
```

Expected result: the ingress gateway rewrites `/api/livez` to the service's `/livez` endpoint and returns `200`. This is the Kubernetes equivalent of Lab 4's external `/api` boundary.

Step 8.5: make mesh traffic policy explicit

Problem: a slow or failing payment Pod keeps receiving traffic and holds connections, and the application alone must absorb the damage.

Pattern: mesh policy limits connections, ejects Pods that return repeated `5xx` errors, and sets a proxy timeout. This protects the platform before the application's own breaker opens.

```
# k8s/istio/payment-policy.yaml
# Payment DestinationRule (mTLS, connection limits, outlier detection)
        and VirtualService timeout.
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata: { name: payment-service }
spec:
  host: payment-service
  trafficPolicy:
    tls: { mode: ISTIO_MUTUAL }
    connectionPool:
      http: { http1MaxPendingRequests: 100, maxRequestsPerConnection:
        20 }
    outlierDetection:
      consecutive5xxErrors: 5
      interval: 5s
      baseEjectionTime: 30s
---
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata: { name: payment-service }
spec:
  hosts: [payment-service]
  http:
    - timeout: 1s
      route: [{ destination: { host: payment-service } }]
```

This is circuit-breaking-like **traffic protection** at the proxy layer. It does not replace application-level business handling: FastAPI still decides whether to queue the order, compensate, or report a retryable error.

Verify the policy is valid:

```
istioctl analyze -n orders   # expect no errors for DestinationRule or
        VirtualService
kubectl -n orders get destinationrule payment-service -o
        jsonpath='{.spec.trafficPolicy.outlierDetection}'; echo
```

Failure clinic

Symptom	Why	Fix
Pods restart continuously during a database outage	liveness probe depends on external DB	liveness checks process; readiness can fail dependency gating if appropriate
`OOMKilled` even though CPU looks fine	memory limit too low / unbounded concurrency	set realistic requests/limits, profile memory, bound queues and fan-out
mTLS errors after mesh enablement	one workload is not enrolled or policy mode mismatches	inspect sidecar injection, peer authentication and destination TLS mode
503 `UF` / `upstream reset` from Envoy	proxy timeout, connection pool, or endpoints issue	check Envoy access logs/metrics plus

Symptom	Why	Fix
		Kubernetes endpoints—not only app logs
retry storm after mesh rollout	both app and mesh retry	assign retry ownership per call path and measure attempts

Capstone — Run a controlled failure game day

Work in pairs. For each scenario, state (1) user-visible behavior, (2) expected logs/metrics/traces, (3) automatic recovery, and (4) operator action.

Injected failure	Expected design behavior
Send identical `POST /orders` three times concurrently	one persisted order; same order ID returned; no duplicate event/business action
Payment waits 5 seconds	order calls fail within deadline; bulkhead protects other requests; breaker opens; no duplicate charge
kill service after DB commit before publish	outbox publisher later emits the event
deliver `OrderCreated` twice	consumer dedupe makes one shipment/notification
shipping fails after authorization	saga voids payment and releases stock, or marks compensation pending for retry
restart every order Pod	requests continue through ready replicas; durable order/saga state survives
roll out a bad payment version	mesh/gateway canary or rollback limits blast radius; telemetry identifies increased 5xx

Completion rubric

- Every unsafe command has idempotency semantics.
- Every remote call has timeout, classification, and bounded concurrency.
- Events are durably recorded before asynchronous publication.
- Consumers are safe under duplicate delivery and expose poison-message handling.
- Each saga step and compensation is durable and idempotent.
- Read-model eventual consistency is explicit to callers.
- The app is stateless; health probes are correctly scoped.
- Gateway, Kubernetes, and mesh responsibilities are not duplicated blindly in FastAPI.
- Logs, metrics, and traces answer: “which order, which saga state, which downstream call, and what failed?”

Instructor notes and extension path

Suggested checkpoints

1. **After Lab 2:** ask why storing order state in Redis alone is unsafe.
2. **After Lab 3:** ask why "retry on any error" can turn a partial outage into a full outage.
3. **After Lab 5:** ask why Kafka does not give business exactly-once processing by itself.
4. **After Lab 6:** ask learners to write compensations before forward steps.
5. **After Lab 8:** ask what should be app code versus a platform policy.

Production additions deliberately left as follow-on labs

- OAuth2/OIDC and service-to-service workload identity; authorization policies per command
- distributed tracing with OpenTelemetry collector, structured logging and Prometheus/Grafana alerting
- consumer schema registry and contract tests
- database migration rollout/rollback strategy and expand–migrate–contract changes
- rate limiting with a distributed store, abuse controls, WAF, and tenant quotas
- OpenAPI generation/client SDKs, load tests (k6/Locust), chaos experiments, and SLO/error-budget design
- a durable workflow engine such as Temporal for long-running sagas that exceed the scope of one service

A pragmatic decision guide

Need	Start with	Add only when
reliable API write	Postgres transaction + idempotency key	always
notify other services	transactional outbox + at-least-once consumer	always for cross-service events
distributed work	explicit saga	there is a true multi-service business transaction
read scaling/query flexibility	CQRS projection	command data model no longer serves query needs
immutable audit/replay	event sourcing	audit/time-travel is primary, not merely interesting
service-to-service protection	Kubernetes Service + timeouts	mesh when platform can operate it well and cross-cutting policy must be uniform

The core lesson is not "use every pattern." It is to make each failure mode explicit, select the smallest mechanism that contains it, and prove it under controlled failure.

Advanced Production Extension — From Reliable Services to Operable Systems

The original one-day workshop covers the core request, data, and workflow patterns. The following **six hands-on extension labs** turn it into a two-day course and address the operational patterns that determine whether a distributed system remains healthy under real traffic, bad deploys, and partial outages.

Advanced-day plan (Labs 9-14)

Time	Lab	Outcome
09:30-10:25	9. Admission control	protect critical traffic during a burst
10:25-11:20	10. Event operations	recover from poison messages and evolve contracts
11:30-12:20	11. Observability and SLOs	trace user impact across sync and async work
13:15-14:05	12. Zero trust	enforce user, workload, and tenant boundaries
14:05-14:55	13. Safe delivery	deploy compatible changes and recover safely
15:05-15:55	14. Data scale and recovery	manage caches, ownership, partitions, and DR
15:55-16:30	Advanced capstone	operate through a traffic and release incident

Extension lab	Pattern added	Failure it prevents
9	Admission control, rate limiting, load shedding, queue-based load leveling	a traffic burst exhausts every worker and downstream dependency
10	Retry topics, DLQ, schema evolution, contract tests	a poison event blocks a consumer or a producer deployment breaks consumers
11	Observability, SLOs, alerting	engineers cannot identify which dependency or tenant caused a failure
12	Zero trust, service identity, authorization policy	an internal caller forges identity or reaches an operation it must not use
13	Safe delivery, feature flags, expand-migrate-contract	a backward-incompatible release or migration causes downtime
14	Cache-aside, data ownership, partitioning, DR decisions	database overload, cache stampede, shared-database coupling, or unclear recovery design

Lab 9 — Admit only the work the system can safely handle

Problem: circuit breakers protect an unhealthy downstream service, but a surge of valid requests can still fill every worker, connection pool, and queue before the breaker opens. A healthy system needs a deliberate answer to: *which work do we accept now, defer, or reject?*

Step 9.1: separate three controls

Problem: a tenant can stay under its request quota and still send sixty expensive operations at once. Rate limiting alone does not protect capacity.

Pattern: three separate controls: a rate limit for fairness per tenant, a concurrency limit for expensive work, and load shedding for non-critical requests when the system is under pressure.

Control	Decision	Typical location
Rate limit	"Has this tenant/user exceeded an agreed quota?"	gateway or Redis-backed limiter
Concurrency limit	"Can this service start one more expensive operation?"	service/mesh worker pool
Load shedding	"Is protecting the system more important than accepting this non-critical request?"	gateway and service

Do not treat them as the same thing. A tenant can remain within its 60 requests/minute quota yet still trigger 60 expensive LLM, payment, or database operations simultaneously.

Verify the burst behavior: send 200 requests with 50 in parallel. The gateway quota should return some `429`s, and the total must never reach the service faster than the limit allows.

```
seq 200 | xargs -P 50 -I{} curl -s -o /dev/null -w "%{http_code}\n"
        localhost:8000/api/livez | sort | uniq -c
# expect 200s up to the quota and 429s for the rest
```

Step 9.2: apply a distributed rate limit at the gateway

Problem: an in-memory counter in each gateway replica gives each tenant a different effective quota depending on which replica receives the request.

Pattern: one shared bucket per tenant in Redis, keyed by the credential that identifies the tenant, so every replica enforces the same limit.

Use the gateway or Envoy rate-limit service for the first protection layer. The key must represent a real fairness boundary: tenant ID, API key, user ID, or client application—not just source IP.

```
# Conceptual gateway policy: one shared Redis-backed bucket per tenant
plugins:
  - name: rate-limiting-advanced
    config:
      limit: [120]
      window_size: [60]
      identifier: credential
      strategy: redis
      retry_after_jitter_max: 3
```

Return `429 Too Many Requests` with `Retry-After`. Never silently queue an interactive request for minutes; callers need an explicit asynchronous job API for that experience.

Verify the rejection contract: once the tenant quota is exhausted, the response is `429` and includes `Retry-After` (when the Redis-backed policy is configured). Confirm with headers:

```
curl -s -D - -o /dev/null localhost:8000/api/livez | grep -iE
        "^(HTTP|retry-after)"
# after the quota is exhausted: HTTP/1.1 429 and a Retry-After line
```

Step 9.3: bound expensive in-process work

Problem: unlimited expensive work fills every worker, connection pool, and queue, and even the health endpoint slows down.

Pattern: a semaphore caps concurrent expensive operations. The time budget applies only to waiting for a slot, so a request that cannot get capacity is refused quickly with `503` and `Retry-After`. Once a slot is held, the operation is not cancelled by the wait budget.

```python
# services/order_service/app/application/admission.py
# Bound expensive work with a semaphore; refuse with 503 when no slot
        frees up in time.
import asyncio
from fastapi import HTTPException

expensive_work_slots = asyncio.Semaphore(25)

async def run_expensive_work(operation):
    try:
        async with asyncio.timeout(0.05):  # budget for waiting, not
        for the operation
            await expensive_work_slots.acquire()
    except TimeoutError:
        # The service is deliberately refusing work before it causes a
        wider outage.
        raise HTTPException(503, "System busy. Retry shortly.",
        headers={"Retry-After": "2"})
    try:
        return await operation()
    finally:
        expensive_work_slots.release()
```

Verify the admission limit: create `tests/test_admission.py`. Forty callers each run a 0.2-second operation: 25 get slots, and the other 15 wait longer than the 0.05-second budget.

```python
# tests/test_admission.py
import asyncio

import pytest
from fastapi import HTTPException

from services.order_service.app.application.admission import
        run_expensive_work


async def slow_operation():
    await asyncio.sleep(0.2)
    return "ok"


@pytest.mark.asyncio
async def test_excess_work_is_refused_with_503():
    results = await asyncio.gather(
        *(run_expensive_work(slow_operation) for _ in range(40)),
        return_exceptions=True,
    )
    refused = [r for r in results if isinstance(r, HTTPException)]
    succeeded = [r for r in results if r == "ok"]
    assert len(succeeded) == 25
    assert len(refused) == 15
    assert all(r.status_code == 503 for r in refused)
```

```
uv run pytest tests/test_admission.py -v
# expect PASSED: 25 succeed, 15 receive 503
```

Use a separate limit per dependency or workload class. Checkout may deserve reserved capacity; report generation or a bulk re-index request should not consume it.

Verify the time budget is applied to waiting only. Run the `test_admission.py` test above, then change the operation to 0.01 seconds: all 40 calls must succeed, because no caller waits longer than the budget.

Step 9.4: turn non-interactive work into a queue

Problem: a 100,000-order export keeps an HTTP connection open for minutes, ties up a worker, and times out on the client.

Pattern: persist the job, return `202` with a status URL immediately, and let queued workers drain the work at a rate they can sustain.

Example: a customer requests a 100,000-order export. Do not keep an HTTP connection open while the API scans data. Persist a job, publish `ExportRequested`, and return `202` plus a job URL.

```
# Export endpoint returns 202 and enqueues work; worker drains at a
        fixed rate.
@router.post("/exports", status_code=202)
async def request_export(body = Body(...)):
    job_id = create_export_job(body)       # durable database row:
        QUEUED
    publish_outbox_event("ExportRequested", {"job_id": job_id})
    return {"job_id": job_id, "status_url": f"/v1/exports/{job_id}"}

# Worker: limited consumer concurrency gives the system a sustainable
        drain rate.
async def handle_export(event):
    mark_job_running(event["job_id"])
    file_url = await build_export_file(event["job_id"])
    mark_job_complete(event["job_id"], file_url)
```

This is **queue-based load leveling**. The queue absorbs a burst; worker concurrency controls drain rate. It is not a license for infinite backlog: monitor oldest message age and reject/defer work before promised completion times become impossible.

Verify the request path returns quickly: enqueueing must not depend on job size. Time the request, then check the job status.

```
time curl -s -o /dev/null -w "%{http_code}\n" -X POST
        localhost:8000/v1/exports \
  -H 'content-type: application/json' -d '{"tenant_id":"tenant-
        a","rows":100000}'
# expect 202 in milliseconds, not seconds
```

Failure clinic

Symptom	Cause	Fix
every Pod is at 100% CPU and even `/livez` is slow	unlimited expensive work entered the service	local concurrency limits, gateway quotas, and fast load shedding
rate limit resets when a gateway replica restarts	in-memory gateway counter	use Redis or a dedicated distributed rate-limit service for a global tenant quota

Symptom	Cause	Fix
export queue grows for hours	arrival rate exceeds worker drain rate	autoscale consumers on lag, cap accepted jobs, communicate expected completion, add per-tenant fairness
one tenant consumes all workers	single global queue and FIFO scheduling	partition quotas/queues by tenant or priority; reserve capacity for critical work
clients retry `503` immediately	no retry contract or jitter	send `Retry-After`, document exponential backoff with jitter, make commands idempotent

Exercise: submit 100 export jobs while workers process two at a time. Track queue depth, oldest-message age, rejection count, and completion latency. Decide at what queue age the API should stop accepting new low-priority exports.

Lab 10 — Make event-driven systems recoverable and evolvable

Problem: an event can fail forever because of corrupt data, an unavailable third party, or a bug. A producer also evolves faster than its consumers. Without bounded retries, dead-letter handling, and compatibility rules, one message can stall a partition or one release can break every consumer.

Step 10.1: use an explicit event envelope

Problem: events are shaped by whatever the producer happened to serialize. Consumers cannot dedupe, trace, or route them reliably, and a field rename breaks everyone.

Pattern: a fixed envelope (ID, type, schema version, time, correlation ID, data) that is versioned as a public contract.

```
{
  "event_id": "8bd1...",
  "event_type": "OrderCreated",
  "schema_version": 1,
  "occurred_at": "2026-10-29T10:00:00Z",
  "correlation_id": "request-7af...",
  "data": {
    "order_id": "o-123",
    "customer_id": "c-456",
    "total": "49.90"
  }
}
```

The envelope gives consumers the fields they need for dedupe, tracing, routing, and version selection. It is a public contract between independently deployed services—not a Python object serialized for convenience.

Step 10.2: classify failures before retrying

Problem: retrying every exception keeps a poison message in an infinite loop, and blocking the consumer with `sleep` stalls the whole partition.

Pattern: classify each outcome. Retry only transient failures, a bounded number of times, using delayed retry topics. Send everything else to a dead-letter queue (DLQ) for repair and replay.

```
# Classify failures to decide success, retry, DLQ, or unknown
        handling.
async def process_message(event):
    try:
        await create_shipment(event["data"])
        return "success"
    except InventoryTemporarilyUnavailable:
        return "retry"
    except InvalidAddress:
        return "business_failure"
    except Exception:
        return "unknown_failure"

async def consume(event, attempt):
    outcome = await process_message(event)
    if outcome == "success":
        return
    if outcome == "retry" and attempt < 5:
        await publish("orders.retry.5m", {"event": event, "attempt":
        attempt + 1})
        return
    await publish("orders.dlq", {
        "event": event,
        "attempt": attempt,
        "reason": outcome,
    })
```

Use delayed retry topics/queues such as `retry.1m`, `retry.5m`, and `retry.30m`; do **not** block a Kafka consumer with `sleep(300)`. A DLQ is not a trash can. It needs an owner, dashboard, alert threshold, repair workflow, and a safe replay tool.

Verify the DLQ route: publish one event that always fails the classification (for example, an invalid address), then read the DLQ topic. It must appear after the bounded attempts, with its reason.

```
docker compose -f infra/compose.yaml exec -T redpanda rpk topic
        consume orders.dlq -n 1
# expect a record containing "reason": "business_failure"
```

Step 10.3: evolve additively first

Problem: a producer renames or removes a field during a rolling deploy. Old consumers still running on other Pods fail, and the failure appears only after the deploy.

Pattern: change contracts additively. New fields are optional, consumers ignore unknown fields, and only a true semantic break gets a new version.

Safe default rules:

- Add optional fields; do not rename, delete, or change the meaning of existing required fields.
- Consumers ignore unknown fields and supply defaults for omitted optional fields.
- Version an event only for a genuine incompatible semantic change.
- Keep old producers/consumers working during the deployment overlap.

```
# Additive schema versioning: normalize v1 and v2 OrderCreated events.
def normalize_order_created(event):
    version = event.get("schema_version", 1)
    data = event["data"]
```

```
    if version == 1:
        # Version 1 did not include currency; preserve the documented
        default.
        data["currency"] = "USD"
    if version not in [1, 2]:
        raise ValueError("Unsupported OrderCreated version")
    return data
```

Step 10.4: test the contract in CI

Problem: a change that looks safe in review still breaks a consumer, and nobody notices until production.

Pattern: keep real historical events as fixtures and replay them through the current consumer code in CI.

```
# Contract tests that replay historical event fixtures through the
        consumer.
# Keep representative historical events in tests/contracts/.
async def test_shipping_accepts_order_created_v1():
    event = load_json("tests/contracts/order-created-v1.json")
    data = normalize_order_created(event)
    assert data["currency"] == "USD"

async def test_shipping_accepts_order_created_v2():
    event = load_json("tests/contracts/order-created-v2.json")
    data = normalize_order_created(event)
    assert data["currency"] == "INR"
```

At larger scale use a schema registry (Avro, Protobuf, or JSON Schema) and block incompatible producer changes in CI. The schema registry does not replace semantic tests: a field can remain syntactically compatible while its business meaning changes.

Verify the contract tests: save `normalize_order_created` and both tests in `tests/test_order_created_contract.py`. Create `tests/contracts/order-created-v1.json` (no `currency`) and `tests/contracts/order-created-v2.json` (`"currency": "INR"`), then run:

```
uv run pytest tests/test_order_created_contract.py -v
# expect 2 PASSED
```

Failure clinic

Symptom	Cause	Fix
a partition has growing lag and one event is logged repeatedly	poison event causes infinite retry	bounded retries, retry topic, DLQ, operator repair/replay flow
DLQ grows unnoticed	no ownership/SLO	alert on DLQ count and age; assign a service owner and runbook
new producer breaks old consumer	renamed/deleted field during rolling deploy	additive change, versioned event, compatibility tests
replay creates duplicate shipments	replay tool bypasses normal consumer dedupe	keep event IDs and idempotency checks active for all replays
retry message changes order	retry recomputes data from the current database	retry the original immutable command/event with an operation ID

Lab 11 — Observe request paths, not isolated services

Problem: logs say “payment timeout,” but no one can answer which customer request caused it, whether it affected one tenant or every request, how often it occurs, or whether the user-visible objective is violated.

Step 11.1: carry a trace across every hop

Problem: logs show “payment timeout” in one service, but nothing ties that line to the order request, the gateway call, or the saga step that caused it.

Pattern: OpenTelemetry spans propagate a `traceparent` header across HTTP and Kafka, so every hop records the same trace ID.

```
# services/order_service/app/observability.py
# Configure OpenTelemetry tracing for FastAPI and outbound httpx
        calls.
from opentelemetry import trace
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.instrumentation.httpx import
        HTTPXClientInstrumentor
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import
        OTLPSpanExporter

def configure_observability(app):
    provider = TracerProvider()
    provider.add_span_processor(BatchSpanProcessor(OTLPSpanExporter()))
    trace.set_tracer_provider(provider)
    FastAPIInstrumentor.instrument_app(app)
    HTTPXClientInstrumentor().instrument()
```

OpenTelemetry propagates `traceparent` on HTTP calls. Include the current trace/correlation ID in Kafka headers too, then consumer spans can join the same end-to-end order timeline.

Verify propagation: send a request with a known trace ID, then find that trace in your OpenTelemetry backend (an OTLP collector or Jaeger listening on port 4317).

```
curl -s -o /dev/null -H 'traceparent: 00-
        4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01'
        localhost:8000/api/livez
# search the backend for trace ID 4bf92f3577b34da6a3ce929d0e0e4736:
        the order-service span must be present
```

Step 11.2: use structured logs and meaningful metrics

Problem: unstructured logs cannot be filtered by order or outcome, and metrics labeled with IDs overload Prometheus.

Pattern: log key-value events that name the operation and outcome, and record RED metrics (rate, errors, duration) with only low-cardinality labels.

```
# Structured logs and failure logging for payment authorization.
import structlog

log = structlog.get_logger()
```

```python
async def authorize_order(order_id, customer_id, total):
    log.info("payment_authorization_started", order_id=order_id,
        customer_id=customer_id)
    try:
        authorization_id = await payment.authorize(order_id, total)
        log.info("payment_authorization_succeeded", order_id=order_id)
        return authorization_id
    except Exception:
        log.exception("payment_authorization_failed",
        order_id=order_id)
        raise
```

Record **RED metrics** for each API: Rate, Errors, Duration. Record queue depth/oldest age for consumers and breaker state/concurrency for dependencies. Avoid a metric label such as `order_id` or `customer_id`; it creates unbounded time-series cardinality and can overload Prometheus.

Verify the labels: after exposing `/metrics` (the `prometheus-fastapi-instrumentator` dependency provides it), no metric may carry an ID label.

```
curl -s localhost:8000/metrics | grep -c 'order_id='   # expect 0
```

Step 11.3: define service-level objectives

Problem: alerts fire on every 500, so engineers ignore them. Nobody can say whether users are actually affected.

Pattern: an SLO states the target. The error budget is what the target allows to fail, and alerts fire on how fast that budget is being spent.

Example checkout SLO:

Signal	Objective	Alert condition
Availability	99.9% of valid `POST /orders` requests accepted monthly	error-budget burn is 10x normal over 1 hour
Latency	95% accepted in under 700 ms	p95 exceeds 700 ms for 15 minutes
Async completion	99% of accepted orders reach terminal state in under 5 minutes	oldest order saga exceeds 5 minutes
Event health	99.9% events leave outbox in under 60 seconds	outbox lag or DLQ age breaches target

Alert on user impact and fast budget burn—not every single 500. An isolated failed request is a log/tracing event; an alert means action is required now.

Verify the budget math: a 99.9% monthly availability SLO on 1,000,000 valid requests allows 1,000 failures. 1,000 failures over 730 hours (one month) is about 1.4 failures per hour. A burn rate of 10 times that is about 14 failures per hour; your burn-rate alert should page at that rate.

Failure clinic

Symptom	Cause	Fix
each service has a different request ID	correlation IDs are generated per hop	use W3C trace propagation; generate only at the trusted edge if absent
Prometheus memory grows without bound	high-cardinality labels such as user/order ID	retain IDs in logs/traces, not metric labels

Symptom	Cause	Fix
dashboards look healthy while orders stay pending	API metrics omit asynchronous workflow health	add saga-age, outbox-lag, consumer-lag, and DLQ metrics
alerts cause noise and are ignored	alerts are based on raw errors rather than SLO burn	alert on error budget and actionable symptoms

Exercise: force payment timeouts, then use one trace ID to find the gateway request, order span, payment call, breaker state, and final saga outcome.

Lab 12 — Apply zero-trust identity and authorization

Problem: network location is treated as identity. Any Pod that can reach `order-service` can send a forged `X-User-Id` header or invoke a refund endpoint. A service mesh encrypts traffic, but encryption alone does not decide who may do what.

Step 12.1: separate human, workload, and permission identity

Problem: network location is treated as identity. Any Pod that can reach `order-service` can send a forged `X-User-Id` header and act as an admin.

Pattern: verify three identities separately: the end user (token), the calling workload (mTLS identity), and the permission (scopes and tenant policy).

Identity	Example	Verification
End user	customer placing an order	gateway validates OIDC access token
Workload	shipping service consuming `OrderCreated`	mesh workload identity / mTLS certificate
Permission	support agent requests refund	scopes, roles, tenant policy, business authorization

The gateway verifies external JWTs and passes trusted claims only through an authenticated internal path. A sensitive service should also validate the token or receive a short-lived signed internal identity token; do not blindly trust arbitrary headers.

Verify (after replacing the placeholder principal with real token validation): a forged header must not grant access.

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST -H 'X-User-Id: admin'
        localhost:8000/v1/orders/11111111-1111-1111-1111-
        111111111111/cancel
# expect 401 with no valid token
```

Step 12.2: enforce permissions in the application

Problem: checking a scope proves the caller may cancel orders, but not which orders. A support agent from one tenant can cancel another tenant's order.

Pattern: a scope dependency for the operation, plus a tenant predicate on every data lookup, so a foreign resource looks the same as a missing one.

```python
# services/order_service/app/api/security.py
# Scope-based authorization dependency for protected routes.
from fastapi import Depends, HTTPException

def current_principal():
    # Production version: validate issuer, audience, signature and
        expiry using cached JWKS keys.
    # This placeholder assumes a verified gateway token was converted
        to a trusted principal.
    return {"subject": "user-123", "tenant_id": "tenant-a", "scopes":
        ["orders:write"]}

def require_scope(scope):
    def check(principal = Depends(current_principal)):
        if scope not in principal["scopes"]:
            raise HTTPException(403, "Missing required permission")
        return principal
    return check

@router.post("/{order_id}/cancel")
async def cancel_order(order_id, principal =
        Depends(require_scope("orders:cancel"))):
    return await order_service.cancel(order_id,
        principal["tenant_id"])
```

Authorization must include the resource boundary. Having `orders:cancel` is insufficient if a support agent from tenant A can cancel tenant B's order. The domain lookup must constrain by tenant/ownership.

Verify the tenant boundary with a valid token for tenant A and an order that belongs to tenant B:

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST -H "Authorization:
        Bearer $TENANT_A_TOKEN" \
  localhost:8000/v1/orders/<tenant-b-order-id>/cancel
# expect 404: a 403 would reveal that the order exists
```

Step 12.3: restrict network paths with mesh policy

Problem: a compromised or misconfigured workload in the same namespace can call the payment service directly, even though only order-service should.

Pattern: a mesh `AuthorizationPolicy` allows calls only from the identity of the order-service workload, so the network path itself enforces who may call.

```yaml
# Istio AuthorizationPolicy: only order-service may call payment
        /authorizations.
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: payment-only-from-order-service
spec:
  selector:
    matchLabels: { app: payment-service }
  rules:
    - from:
        - source:
            principals: ["cluster.local/ns/orders/sa/order-service"]
      to:
        - operation:
            methods: ["POST"]
            paths: ["/authorizations"]
```

Use a secret manager and workload identity to obtain secrets dynamically. Do not put API keys in images, ConfigMaps, Git, log messages, traces, or exception responses. Rotate secrets and signing keys without downtime.

Verify the mesh policy: a probe running as a different identity must be refused.

```
kubectl -n orders run probe --rm -i --restart=Never --
        image=curlimages/curl -- \
  curl -s -o /dev/null -w "%{http_code}\n" -X POST http://payment-
        service/authorizations
# expect 403 (RBAC denied by the sidecar)
```

Failure clinic

Symptom	Cause	Fix
caller sends `X-User-Id: admin`	service treats a header as proof of identity	validate signed tokens; restrict direct service access
valid JWT for tenant A reads tenant B data	authorization checks scope but not resource ownership	include tenant/ownership predicate in every data access path
mesh mTLS enabled but unauthorized API call succeeds	mTLS authenticates workload, not operation permission	add mesh authorization policy and app-level business authorization
key rotation causes all requests to fail	JWKS/secret cache only knows one key	cache by key ID and retain old verification keys during overlap

Lab 13 — Release safely when services and schemas evolve independently

Problem: deploys are rolling. For some period, old and new Pods, consumers, and database code run together. A release that requires every component to switch at the same instant is not a production release strategy.

Step 13.1: use expand-migrate-contract for database changes

Problem: a rolling deploy runs old and new code at the same time. Dropping or renaming a column in one release breaks the Pods still running the old version.

Pattern: expand the schema first (add, keep the old column), migrate the data and code, and contract (drop) only after no reader depends on the old shape.

Example: replace `orders.customer_email` with `customer_id`.

```
-- Expand-migrate-contract: add the new column, backfill, then drop
        the old one.
-- Release A: EXPAND. Old code still works.
ALTER TABLE orders ADD COLUMN customer_id UUID NULL;

-- Release B: MIGRATE. New code writes both fields; a backfill job
        fills old rows.
UPDATE orders SET customer_id = customer_lookup(email)
WHERE customer_id IS NULL;
```

```sql
-- Release C: CONTRACT, only after every reader uses customer_id and
        verification is complete.
ALTER TABLE orders DROP COLUMN customer_email;
```

Never combine a destructive schema change with the first deployment that stops using it. The same rule applies to event fields and external API versions.

Verify the expand phase: after Release A and before the contract step, run the previous release's `INSERT` against the migrated schema. It must succeed, which shows the expand step is backward compatible.

Step 13.2: use a feature flag for behavior changes

Problem: a new payment path needs to ship with the code but serve traffic only when it is ready, and it must be switched off without a redeploy.

Pattern: a feature flag selects the path per tenant at runtime, with a safe default so a flag-service outage keeps the old behavior.

```python
# Feature flag selects the payment path per tenant with a safe
        default.
async def select_payment_path(order):
    if feature_flags.enabled("payment-v2",
        tenant_id=order["tenant_id"]):
        return await payment_v2.authorize(order)
    return await payment_v1.authorize(order)
```

Feature flags need owners, expiry dates, audit history, and a safe default. A permanent forest of flags becomes hidden distributed configuration and makes incidents harder to debug.

Verify both branches: with a fake flag client, test that the flag off calls `payment_v1.authorize`, and the flag on for one tenant calls `payment_v2.authorize`. Assert which fake was called, not only the result.

Step 13.3: canary traffic before full rollout

Problem: a new release that breaks checkout reaches every customer at once if it is switched on in one step.

Pattern: send a small weight of traffic to the canary subset, compare it with the stable version on errors, latency, and business outcomes, then promote or roll back.

```yaml
# Istio weighted canary split: 95% stable, 5% canary.
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata: { name: order-service }
spec:
  hosts: [order-service]
  http:
    - route:
        - destination: { host: order-service, subset: stable }
          weight: 95
        - destination: { host: order-service, subset: canary }
          weight: 5
```

Promote only after comparing canary versus stable on request errors, latency, outbox lag, DLQ rate, and business outcomes such as payment authorization success. A technical `200` can still represent a broken order workflow.

Verify the traffic split: send 200 requests through the mesh and count responses per subset in Prometheus with `sum by (destination_version) (istio_requests_total{destination_service_name="order-service"})`. At a

5% weight, expect about 10 canary requests.

Step 13.4: drain before termination

Problem: Kubernetes stops a Pod while requests and messages are still in flight. Those requests fail, and some work is processed twice.

Pattern: fail readiness first, so no new traffic arrives, then finish in-flight work within the grace period before the process exits.

```
# Drain: fail readiness, sleep in preStop, and allow a long
        termination grace period.
spec:
  terminationGracePeriodSeconds: 45
  containers:
    - name: app
      lifecycle:
        preStop:
          exec:
            command: ["/bin/sh", "-c", "sleep 10"]
```

The readiness endpoint must turn unhealthy before process exit so Kubernetes and the mesh stop routing new traffic. The application then stops consuming new messages, finishes in-flight work within a bounded time, commits offsets/transactions, and closes connections.

Verify zero errors during termination: keep requests flowing through the ingress (port-forward from Step 8.4) while one Pod is deleted.

```
(for i in $(seq 300); do curl -s -o /dev/null -w "%{http_code}\n"
        localhost:8080/api/livez; sleep 0.1; done) | sort | uniq -c >
        /tmp/drain.txt &
POD=$(kubectl -n orders get pods -l app=order-service -o name | head
        -1)
kubectl -n orders delete "$POD" --wait=false
wait; cat /tmp/drain.txt
# expect only 200 lines: no 5xx while the Pod terminates
```

Failure clinic

Symptom	Cause	Fix
canary has 200s but order completion drops	only HTTP status is checked	promotion metrics include async workflow and business KPIs
migration blocks writes for minutes	destructive DDL on a large table during peak traffic	use online migration techniques, backfill jobs, and expand-migrate-contract
shutdown duplicates work	consumer exits before idempotent result/offset is safely committed	stop intake first, finish bounded work, retain idempotency on restart
feature flag becomes permanent and unknown	no ownership or expiry	require owner, purpose, expiration, and deletion ticket

Lab 14 — Scale data deliberately: caches, ownership, partitions, recovery

Problem: adding replicas is easy until every replica reads the same hot data, cache expiry causes a database stampede, and several services directly modify the same tables. At larger scale, data boundaries—not HTTP routes—determine autonomy and reliability.

Step 14.1: use cache-aside only for safely stale reads

Problem: every catalog read goes to the database, which becomes the bottleneck as traffic grows.

Pattern: cache-aside. Read the cache first, load from the database on a miss, and store the result with a TTL. Use it only where stale data is acceptable.

```
# Cache-aside read with a 60-second TTL.
async def get_catalog_item(sku):
    key = f"catalog:{sku}"
    cached = await redis.get(key)
    if cached:
        return json.loads(cached)

    item = await catalog_repository.get(sku)
    if item is None:
        return None

    await redis.set(key, json.dumps(item), ex=60)
    return item
```

Cache-aside is appropriate for product catalog, configuration, and precomputed summaries. Do not use it as the primary authority for payment balance, stock reservation, or an order's current saga state without carefully defined consistency rules.

Verify the TTL behavior: with an empty cache, make 1,000 reads for one SKU within 60 seconds. The repository must be called once, and once more after the TTL expires.

Step 14.2: prevent cache stampedes

Problem: a popular key expires, and every concurrent request misses the cache and queries the database at the same moment.

Pattern: single-flight. Only one caller per key refreshes the cache; the others wait and then read the fresh value.

```
# Local single-flight lock to prevent stampedes inside one replica.
import asyncio

refresh_slots = {}

async def get_catalog_item_singleflight(sku):
    if await redis.get(f"catalog:{sku}"):
        return await get_catalog_item(sku)

    lock = refresh_slots.setdefault(sku, asyncio.Lock())
    async with lock:
        # Recheck: another request may have populated it while this
        request waited.
        return await get_catalog_item(sku)
```

This local "single-flight" lock prevents a stampede only inside one replica. For cross-replica hot keys, prefer probabilistic early refresh, stale-while-revalidate, or a carefully bounded distributed lease. Never use a distributed lock as the first response to ordinary concurrency; it adds its own availability failure mode.

Verify single-flight: launch 50 concurrent `get_catalog_item_singleflight("SKU-1")` calls against an empty cache. The fake repository's `get` must be called once, not 50 times.

Step 14.3: make ownership explicit

Problem: when services share tables, a schema change in one service silently breaks another, and deployments must be coordinated.

Pattern: each service owns its data. Other services use its API or events, never its tables.

Data	Owning service	How other services use it
Order and saga state	order-service	API or `Order*` events
Stock reservation	inventory-service	reservation API and `Stock*` events
Payment authorization	payment-service	payment API and `Payment*` events
Search/dashboard view	reporting-service	projections fed by events

No service writes another service's database tables. A shared database makes a local schema change a hidden distributed deployment. Use an anti-corruption layer at boundaries: translate external/legacy concepts into your own domain commands rather than allowing their model to leak throughout the system.

Verify ownership: no other service's code may read the order tables.

```
grep -rln "FROM orders" services | grep -v "services/order_service" ||
        echo "no cross-service table access"
# expect: no cross-service table access
```

Step 14.4: choose partitions and regional recovery intentionally

Problem: a backup that has never been restored is not a recovery plan. Without measured RPO (how much data can be lost) and RTO (how long recovery takes), a failover is a guess.

Pattern: choose partitioning and regional strategy from the RPO and RTO you have measured, not from scale alone.

Need	Pattern	Design question
one database is too large/hot	partition/shard by tenant or order ID	can one order ever require a cross-shard transaction?
read traffic dominates	replicas or CQRS projection	how stale may a read be?
single-region outage tolerance	active-passive disaster recovery	what are acceptable RPO and RTO?
global low-latency writes	multi-region ownership/partitioning	which region owns a given order to avoid conflicting writes?

Start with a tested backup and restore, then regional failover drills. Multi-region active-active writes are a product and data-consistency decision, not a switch on a load balancer. Use them only when the RPO/RTO and latency requirements justify the complexity.

Verify with a restore drill: back up, restore into a scratch database, and time it. The restore time is your measured RTO; the newest `created_at` in the restored data shows the data-loss window (RPO).

```
docker compose -f infra/compose.yaml exec -T postgres pg_dump -U
        orders -Fc orders > /tmp/orders.dump
docker compose -f infra/compose.yaml exec -T postgres createdb -U
        orders orders_restore
time docker compose -f infra/compose.yaml exec -T postgres pg_restore
        -U orders -d orders_restore < /tmp/orders.dump
```

Failure clinic

Symptom	Cause	Fix
database CPU spikes after popular cache key expires	every replica misses at once	jitter TTLs, stale-while-revalidate, single-flight/lease, capacity test hot keys
stale catalog price used for payment	cache used for a correctness-critical decision	revalidate authoritative price during order command
services cannot deploy independently	shared tables and cross-service joins	establish service-owned data and event/API boundaries
shard migration becomes an emergency	shard key chosen without growth/rebalancing plan	record routing metadata, design tenant move procedure, test it before scale requires it
disaster recovery plan exists only in a document	backup/restore never exercised	run restore and regional failover game days; measure actual RPO/RTO

Advanced capstone — Operate the platform through a release and traffic incident

Run this scenario in teams:

1. A new shipping consumer expects `delivery_window`, but old `OrderCreated` events do not have it.
2. A promotion creates 20x export traffic from one tenant.
3. Payment latency rises and the canary version has a higher timeout rate.
4. During rollback, an order-service Pod is terminated with work in flight.

The team must:

- Use traces, metrics, queue age, and DLQ dashboards to diagnose the impact.
- Shed/defer low-priority exports while preserving checkout capacity.
- Route poison/old events through the compatibility path or DLQ, then replay safely.
- Halt or roll back the canary based on an explicit promotion threshold.
- Confirm no duplicate charge, duplicate shipment, dropped order event, or cross-tenant access occurred.

Extended completion rubric

- Traffic is admitted according to tenant quota, operation cost, and system capacity.
- Asynchronous failures have retry, DLQ, repair ownership, and replay semantics.
- Event and API contracts survive independent producer/consumer deployments.

- One trace connects ingress, API, database/outbox, messaging, saga, and downstream calls.
- Alerts are tied to SLO/error-budget impact and asynchronous workflow health.
- Identity, authorization, data ownership, and tenant isolation are enforced at more than one layer.
- Every release is compatible with a mixed fleet and can be halted or rolled back safely.
- Cache behavior, partitioning strategy, backup/restore, and regional recovery have explicit correctness and recovery assumptions.

Interactive incident drills

Use these after the labs. Read only the scenario first, decide what you would do, then expand the answer. A strong answer names the **failure boundary**, the **pattern**, the **evidence**, and the **safe user-visible result**.

Drill 1 — "The user says they were charged twice"

Scenario: The gateway retried a `POST /orders` after a network timeout. The order service returned no response to the client. Payment shows two authorization attempts.

▶ Reveal the investigation and design answer

Drill 2 — "Kafka lag rises but API success is 99.99%"

Scenario: `POST /orders` is fast and successful, but confirmation emails and shipping updates are delayed by 45 minutes.

▶ Reveal the investigation and design answer

Drill 3 — "A canary looks healthy, but customers cannot complete checkout"

Scenario: The canary returns HTTP 200 for checkout requests. Twenty minutes later, `PAYMENT_AUTHORIZED` orders remain stuck and shipping has not started.

▶ Reveal the investigation and design answer

Drill 4 — "A support token accesses another tenant's order"

Scenario: The caller has `orders:read`, but the request URL contains an order from a different tenant.

▶ Reveal the investigation and design answer

Drill 5 — Choose the next pattern

Complete the blanks before opening the answer:

```
# Pattern-selection exercise; answers are below.
1. Database commit succeeds, event publish is lost      -> __________
2. Slow payment dependency consumes every worker         -> __________
```

```
3. Consumer fails forever on malformed event           -> _____
4. Dashboard query slows order creation                -> _____
5. Two services must reverse business steps on failure -> _____
6. Cache expiry creates thousands of database reads    -> _____
```

▶ Reveal answers

Self-learning runbook and review checklist

Use this section as your navigation when you return to the workshop after a break. It states what must be running, the proof of completion, and the most likely first diagnostic step.

Start and stop safely

Core local environment: Labs 0-7

```
# Start only infrastructure. FastAPI and mocks run in separate
        terminals.
docker compose -f infra/compose.yaml up -d postgres redpanda

# Start the FastAPI service using the host environment file.
uv run uvicorn services.order_service.app.main:app --env-file .env --
        reload --port 8000

# When finished with core labs, preserve volumes or remove them
        intentionally.
docker compose -f infra/compose.yaml down
# To erase local database/event state as well:
# docker compose -f infra/compose.yaml down --volumes
```

Compose gateway environment: Labs 4-7 and 9-11

```
# Compose gateway stack: build, follow logs, and smoke-test /api.
docker compose -f infra/compose.yaml up --build -d
docker compose -f infra/compose.yaml logs -f kong order-service

# The external boundary is Kong. Use /api, never a direct application
        port.
curl -i http://localhost:8000/api/livez
```

Kubernetes platform environment: Lab 8 and platform parts of Labs 12-13

```
# Kubernetes context and log checks for the orders namespace.
kubectl config current-context
kubectl get namespace orders
kubectl -n orders get pods,svc,hpa
kubectl -n orders logs deployment/order-service -c app --tail=50
kubectl -n orders logs deployment/order-service -c istio-proxy --
        tail=50
```

> [!CAUTION] Never run **docker compose down –volumes** casually. It deliberately deletes the learning database and message state. Use it only when the lab tells you to reset state or when you want to repeat from a clean slate.

Completion evidence by lab

Lab	Minimum evidence you should save	If it fails, inspect first
0	GET /livez returns 200; missing DATABASE_URL causes startup failure	terminal environment and .env path
1	invalid UUID returns 422; blocking endpoint visibly delays concurrent requests	route registration and request body/header
2	same idempotency key returns one order ID; database has one row	orders/idempotency_keys rows and transaction logs
3	injected delay yields bounded timeout; later requests fail fast while breaker is open	mock dependency port, client deadline, breaker state
4	/api/v1/orders returns 202 through Kong; direct application address is absent	Kong config mount, strip_path, Compose DNS
5	crash-after-write scenario leaves unpublished outbox row; publisher later marks it published	outbox table, producer logs, topic key
6	shipping failure results in void/release or COMPENSATION_PENDING	durable order status and operation IDs
7	query model can lag; aggregate version conflict is rejected/retried	projection offset and event version
8	Pod contains istio-proxy; ingress /api/livez returns 200	namespace injection label, image loading, gateway/VirtualService
9	overload returns 429/503 before worker exhaustion; queue age stays measurable	rate-limit key, semaphore limit, consumer lag
10	malformed message ends in DLQ after bounded attempts; replay is idempotent	retry headers, DLQ owner, consumer dedupe table
11	one trace connects ingress to downstream event/saga work	trace propagation headers and metric label cardinality
12	wrong scope or wrong tenant returns 403/404 without leaking data	token validation, tenant predicate, mesh policy
13	canary sees business metrics; migration stays compatible with old code	rollout status, feature-flag default, schema phase
14	cache miss does not stampede database; restore/failover target is documented	TTL policy, key ownership, RPO/RTO exercise

Common ambiguity breakers

Question	Course answer
“Should I call a Pod IP?”	No. Compose service name locally; Kubernetes Service name in cluster.

Question	Course answer
“Should every request go through Kafka?”	No. Keep immediate user commands synchronous when they need an immediate decision; use events for durable cross-service propagation and long-running work.
“Does a circuit breaker guarantee no duplicate payment?”	No. Idempotency at your payment operation boundary does that.
“Does exactly-once Kafka delivery solve business duplicates?”	No. Consumers still require dedupe/idempotent effects.
“Can a mesh replace saga logic?”	No. Mesh controls transport behavior; a saga owns business progress and compensation.
“Is Kubernetes required to learn FastAPI production patterns?”	No. Compose teaches application behavior first; Kubernetes makes platform responsibilities explicit later.
“Should cache be the source of truth?”	Normally no. Use authoritative storage for correctness-critical writes and decisions.

Deliberately simplified versus production-required

Learning simplification	Production requirement before launch
local Postgres password and environment file	managed secret store, rotation, least-privilege database account
mock payment/inventory services	contract tests, sandbox accounts, provider-specific idempotency and reconciliation
local Kong policy	distributed tenant quota, WAF, auth plugin, gateway SLOs
short in-process publisher example	supervised worker/CDC, deployment lifecycle, topic ACLs, schema registry
placeholder Kubernetes dependency URLs	managed dependencies, real readiness policy, backup and restore validation
illustrative authorization principal	OIDC issuer/audience/JWKS validation, workload identity, resource/tenant authorization

If a step uses a simplification, the table tells you the next production question to ask. This prevents the common mistake of treating a workshop shortcut as an enterprise architecture recommendation.